Big data analytic
Clustering

Le Thanh Van

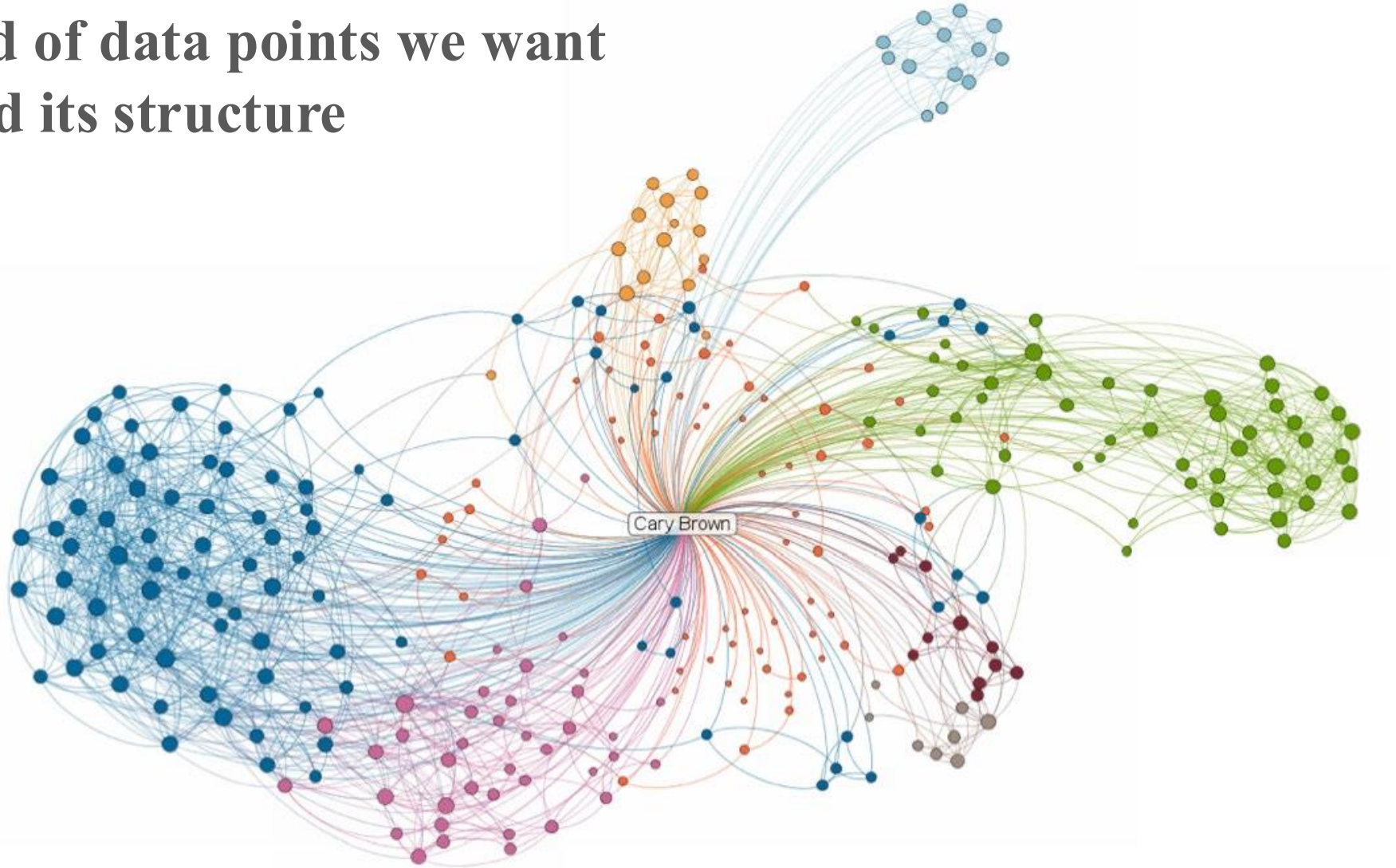
HCMC University of Technology

Acknowledgement

- The lecture uses a portion of slides from “Mining of massive datasets” copyrighted by Jure Leskovec, Anand Rajaraman, Jeff Ullman, Stanford University.
- http://www.mmds.org

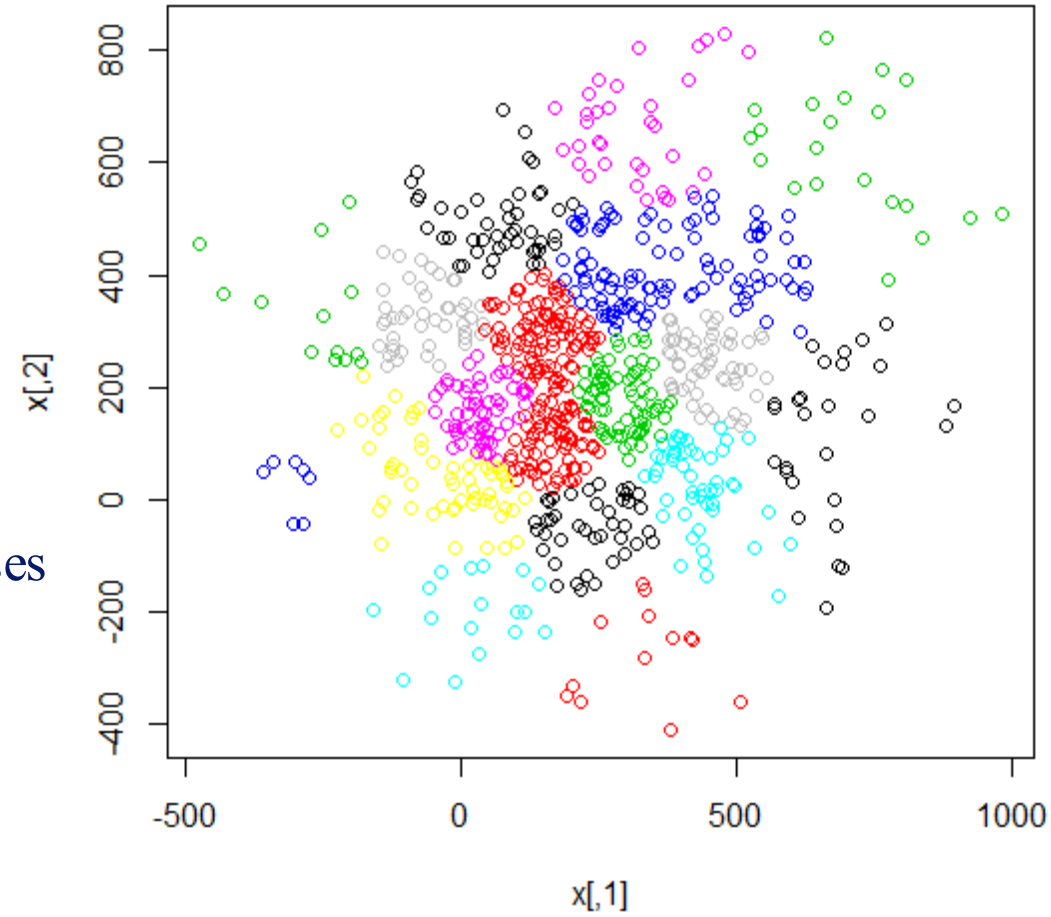
High dimensional data

Given a cloud of data points we want to understand its structure



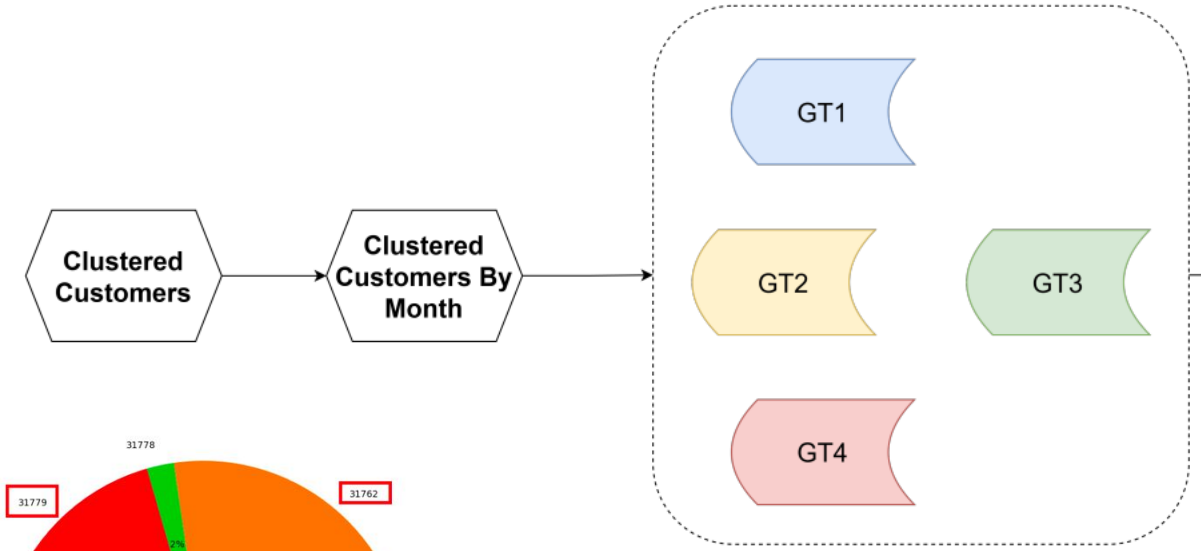
What is Cluster Analysis?

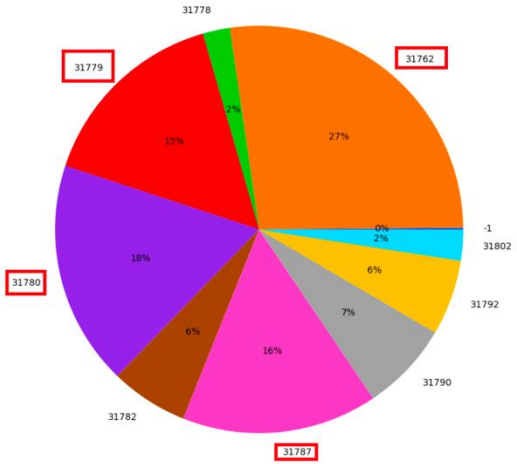
- Cluster: a collection of data objects
  - Similar to one another within the same cluster
  - Dissimilar to the objects in other clusters
- Similarity is defined using a distance measure
  - Euclidean, Cosine, Jaccard, edit distance, …
- Cluster analysis
  - Grouping a set of data objects into clusters
- Clustering is unsupervised classification: no predefined classes
- Typical applications
  - As a stand-alone tool to get insight into data distribution
  - As a preprocessing step for other algorithms

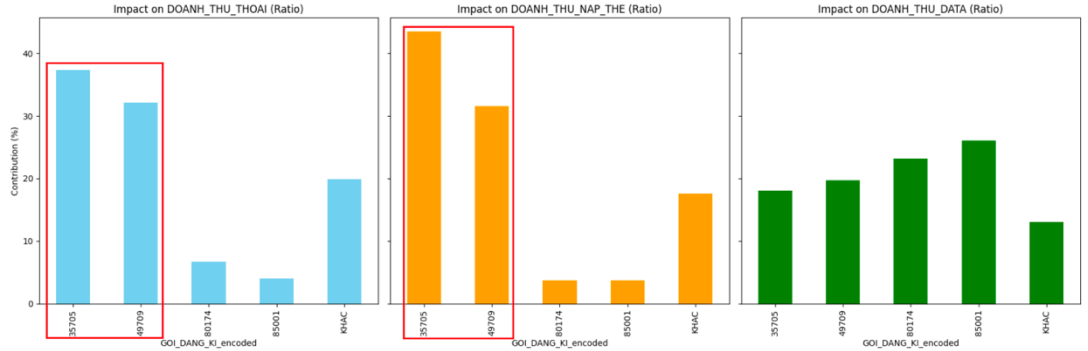


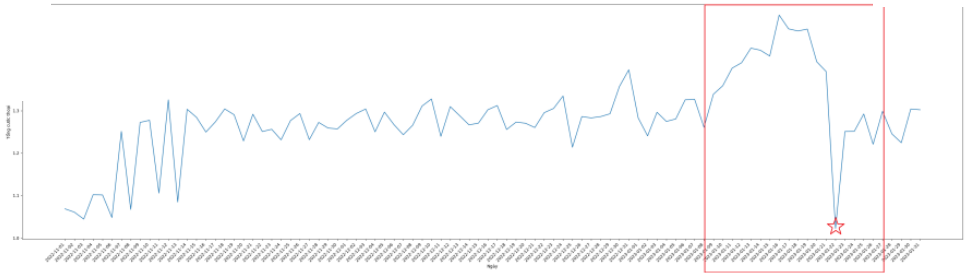
An example

Predicting Customer Spending on Telecommunication Services Based on Information Extracted from Cluster Analysis

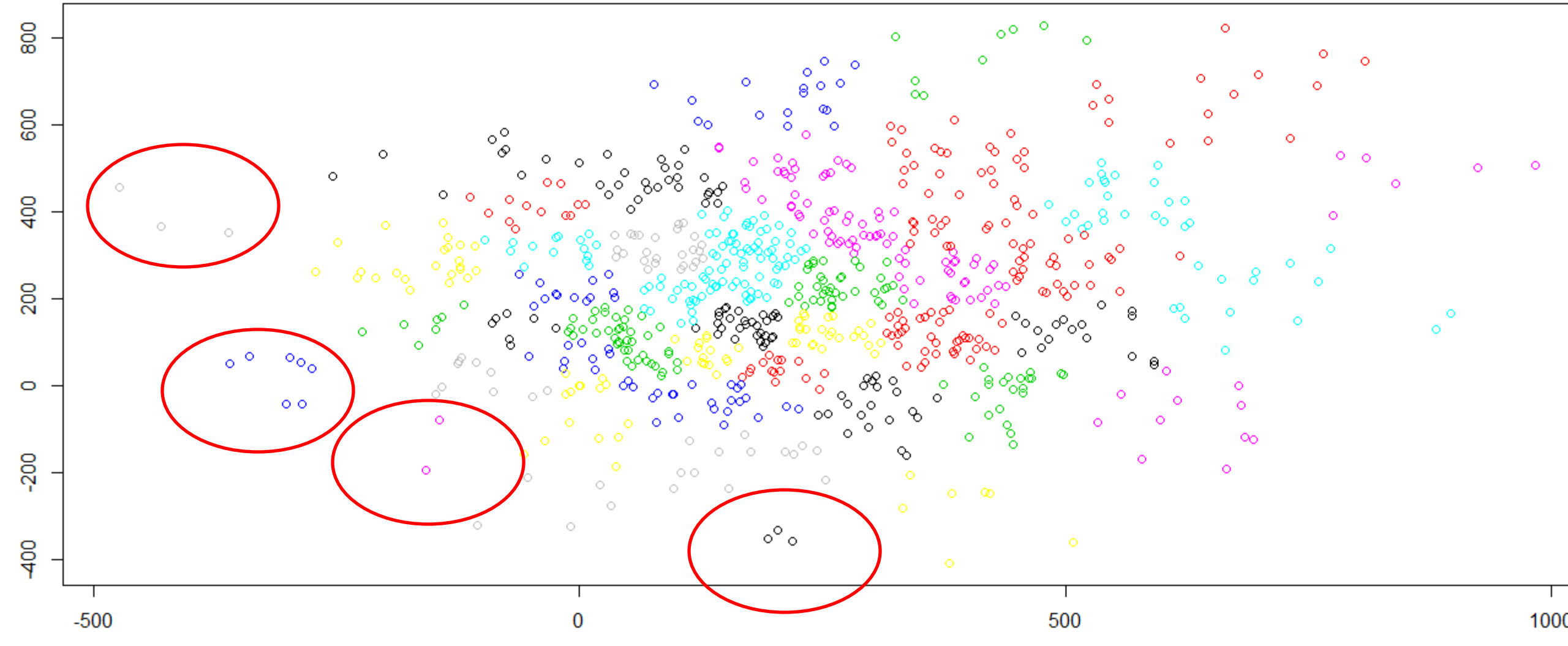




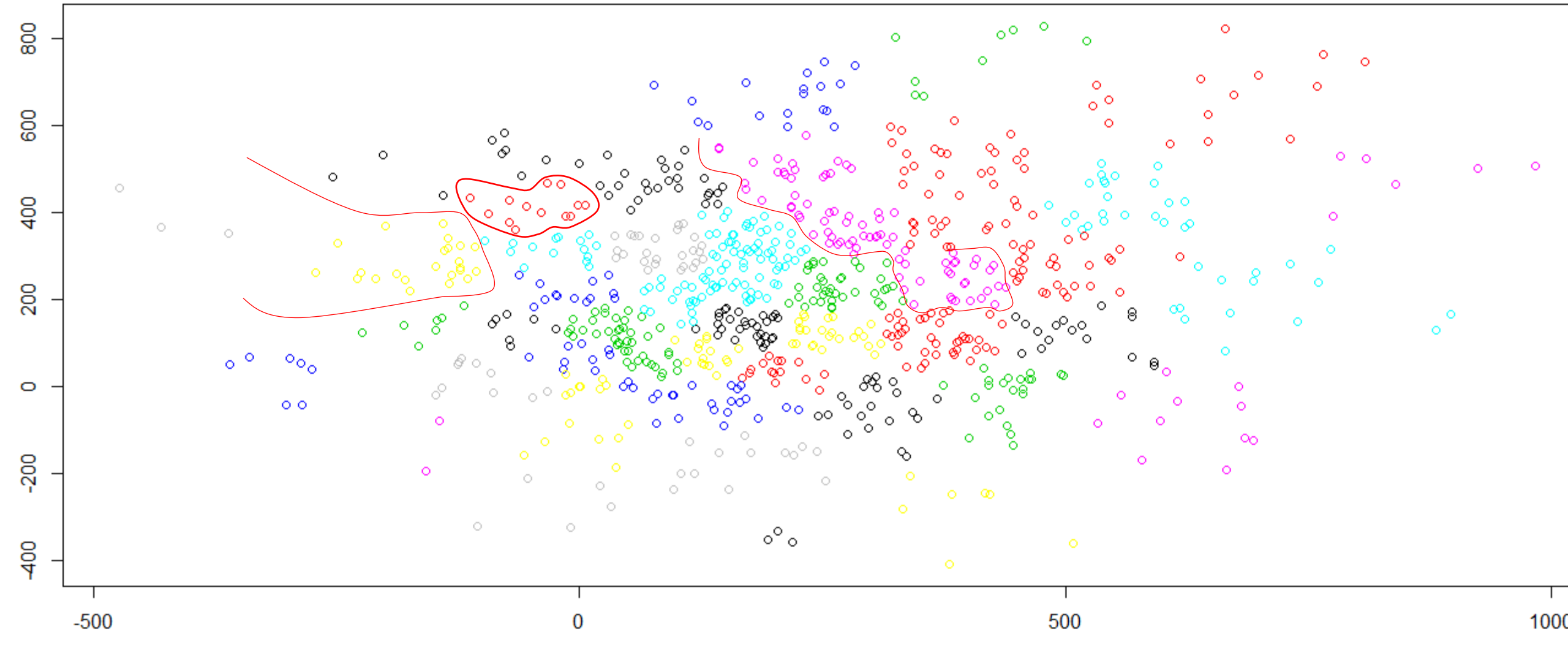




Clusters and outliers



Clustering is a hard problem



Why is it hard?

- Clustering in two dimensions looks easy
- Clustering small amounts of data looks easy
- Many applications involve not 2, but 10 or 10,000 dimensions
- **High-dimensional spaces look different:**
  - Almost all pairs of points are at about the same distance

Clustering Problem: Galaxies

- **A catalog of 2 billion "sky objects" represents objects by their radiation in 7 dimensions (frequency bands)**
- **Problem: Cluster into similar objects, e.g., galaxies, nearby stars, quasars, etc.**
- **Sloan Digital Sky Survey**

Clustering problem: Music CDs

- **Intuitively: Music divides into categories, and customers prefer a few categories**
  - But what are categories really?
- Represent a CD by a set of customers who bought it
  - **Similar CDs have similar sets of customers, and vice-versa ???**
- **Space of all CDs:**
  - Think of a space with one dim. for each customer Values in a dimension may be 0 or 1 only
  - A CD is a point in this space (x_1, x_2,…, x_k), *where* x_i = *1 iff the* i^{th} *customer bought the CD*
- For Amazon, the dimension is tens of millions
- **Task: Find clusters of similar CDs that have similar sets of customers**

Clustering problem: Documents

- **Finding topics:**
  - Represent a document by a vector (x_1, x_2, …, x_k), *where* x_i = *1 iff the* i^{th} *word (in some order) appears in the document*
    - *It actually doesn't matter if k is infinite; i.e., we don't limit the set of words*

- **Documents with similar sets of words may be about the same topic**

Document clustering based on abstracts, tags

----original document data----			
id	-	abstract	"tag"
E43174127280T162	-	Approximating points by piecewise linear fur	"Points approximationÂ â€“Â Computational geometryÂ â€“Â Algorithm and data structureÂ â€“Â Outliers "
----end original document data----			
----begin mcp list data----			
id	score	abstract	"tag"
X88J726U360W9626	0.155984-0.0980581	A set S ? V is a power dominating set (PD	"DominationÂ â€“Â Power dominationÂ â€“Â Interval graphsÂ â€“Â Circular-arc graphsÂ â€“Â Algorithm "
D606716153485175	0.146771-0.102062	The longest path problem is the problem of fi	"Longest path problemÂ â€“Â Cocomparability graphsÂ â€“Â Permutation graphsÂ â€“Â Polynomial algorithmÂ â€“Â Complexity
BR4472X032601267	0.106983-0.288675	A star graph is a tree of diameter at most t	"Approximation algorithmsÂ â€“Â Spanning star forest problem "
94682R0327K24875	0.160544-0.0944911	This paper is concerned with online schedulir	"Online algorithmsÂ â€“Â Competitive analysisÂ â€“Â SchedulingÂ â€“Â Energy efficiencyÂ â€“Â Dynamic speed scalingÂ â€“Â
J661868010732545	0.0320384-0.0833333	Suppose that T is a spanning tree of a grapl	"AlgorithmÂ â€“Â Circular-arc graphÂ â€“Â Interval graphÂ â€“Â Locally connected spanning treeÂ â€“Â Spanning tree "
E43174127280T162	0.0-0.0	Approximating points by piecewise linear fur	"Points approximationÂ â€“Â Computational geometryÂ â€“Â Algorithm and data structureÂ â€“Â Outliers "
MWN4Q223T13H4JW6	0.313223-0.213201	In this paper, we consider multi-objective evo	"Evolutionary algorithmsÂ â€“Â Fixed-parameter tractabilityÂ â€“Â Vertex coverÂ â€“Â Randomized algorithms "
X3V3452732Q180J7	0.172222-0.0	Let (V , ?) be a finite metric space, where	"Directed graphsÂ â€“Â SpannersÂ â€“Â Disk graphs "
Y76187170765L1U6	0.157834-0.5	In the Flow Edge-Monitor Problem, we are g	"Approximation algorithm "
J282J5138Q02834T	0.110297-0.283473	In this paper we investigate dynamic speed s	"SchedulingÂ â€“Â Dynamic speed scalingÂ â€“Â Energy efficiencyÂ â€“Â NP-hardnessÂ â€“Â Approximation algorithmÂ â€“Â
76L4245013129P34	0.120213-0.125	We consider online competitive algorithms fo	"Online algorithmsÂ â€“Â Competitive analysisÂ â€“Â Packet schedulingÂ â€“Â Buffer management "
35U9496J6P29656H	0.0248582-0.0495074	In this paper, we investigate three strategies	"Graph algorithmsÂ â€“Â Navigating in graphsÂ â€“Â Spanning treesÂ â€“Â Routing in graphsÂ â€“Â DistancesÂ â€“Â Ancestry
8841U6238M66N238	0.178146-0.288675	We consider the online smoothing problem ,	"Online algorithmsÂ â€“Â Randomized algorithms "
A3M54P1445761464	0.181691-0.283473	We study three complexity parameters that,	"Intersection graphsÂ â€“Â Parameterized approximation algorithmsÂ â€“Â Graph algorithmsÂ â€“Â NP-hard problems "
N20077064029T718	0.136612-0.288675	We introduce and study the donation center	"Approximation algorithmsÂ â€“Â Facility locationÂ â€“Â Matching with preferences "
X2K218845025367Q	0.283666-0.25	We introduce a new framework for designin	"Approximation algorithmsÂ â€“Â Facility locationÂ â€“Â Sub-modular optimizationÂ â€“Â Display advertising "
E11K403131361207	0.140992-0.294174	We investigate a metric facility location prob	"Facility locationÂ â€“Â Distributed algorithmÂ â€“Â Randomized approximation algorithmÂ â€“Â Synchronous message passing r

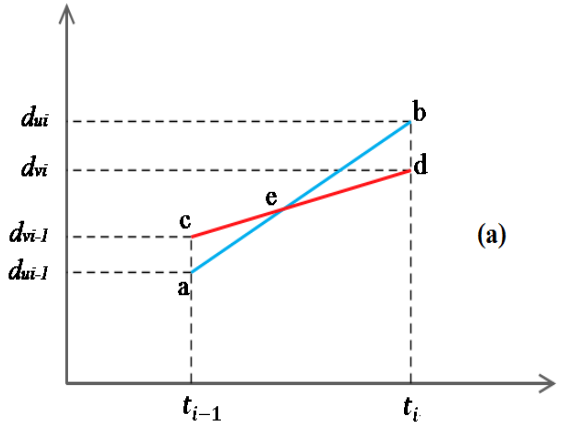
GPS clustering for public traffic

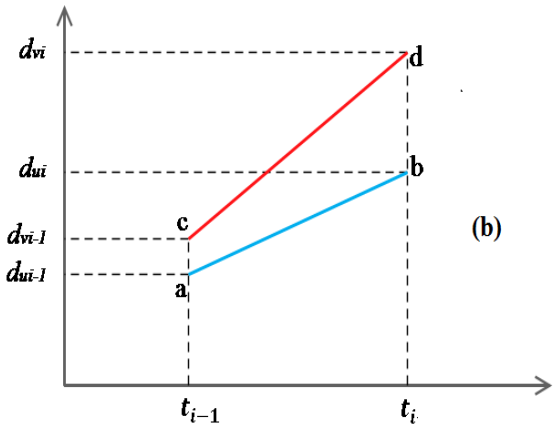
- Area-based profile alignment – ABPA

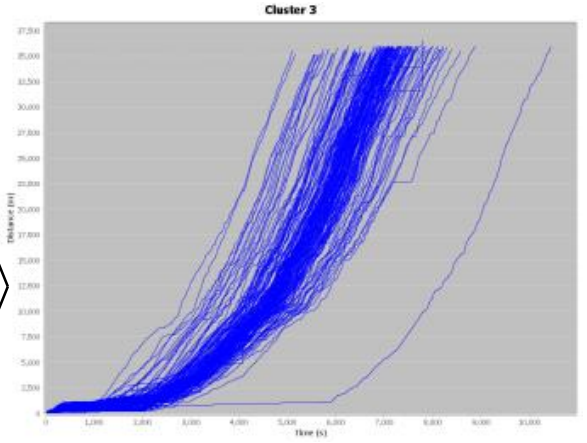
[L. Rueda, 2008]

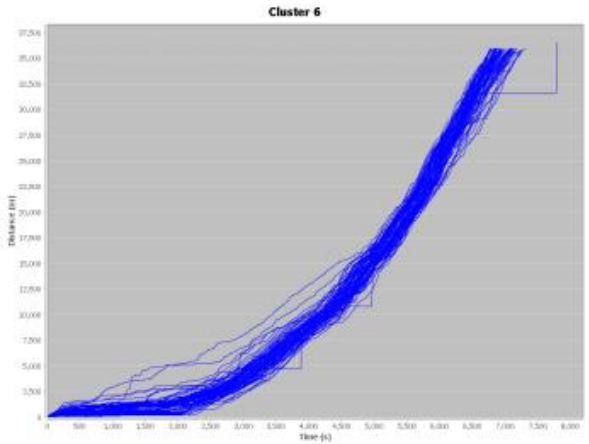
$$U = \langle (d_{u1}, t_{u1}), (d_{u2}, t_{u2}), \ldots, (d_{ui}, t_{ui}), \ldots, (d_{un}, t_{un}) \rangle$$

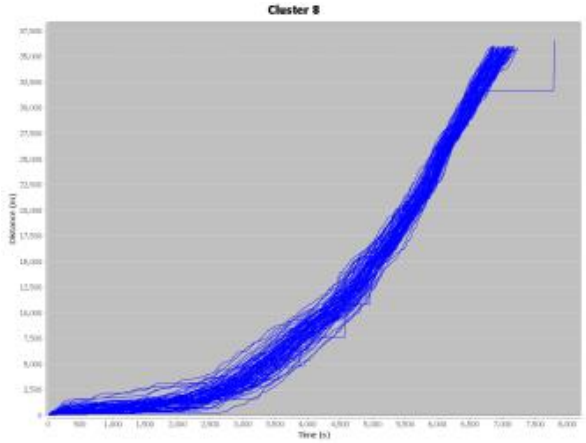
$$V = \langle (d_{v1}, t_{v1}), (d_{v2}, t_{v2}), \ldots, (d_{vi}, t_{vi}), \ldots, (d_{vn}, t_{vn}) \rangle$$

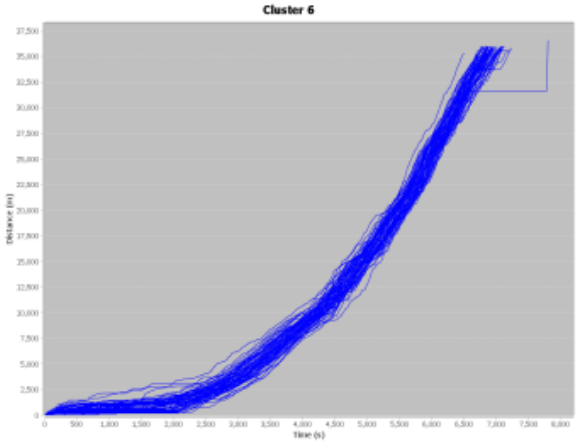












Data Structures

- Data matrix
  - (two modes)

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}$$

- Dissimilarity matrix
  - (one mode)

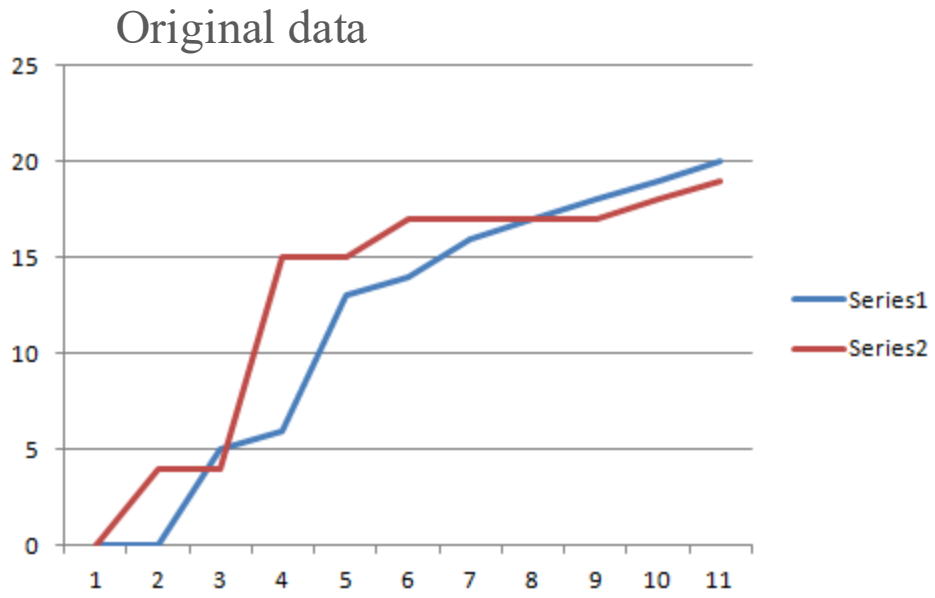
$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \cdots & \cdots & 0 \end{bmatrix}$$

Measure the Quality of Clustering

- Dissimilarity/Similarity metric: Similarity is expressed in terms of a distance function, which is typically metric: $s(i, j)$, $d(i, j)$,
- There is a separate "quality" function that measures the "goodness" of a cluster.
- The definitions of distance functions are usually very different for interval-scaled, boolean, categorical, ordinal and ratio variables.
- Weights should be associated with different variables based on applications and data semantics.
- It is hard to define "similar enough" or "good enough"
  - the answer is typically highly subjective.

Similarity/dissimilarity measure

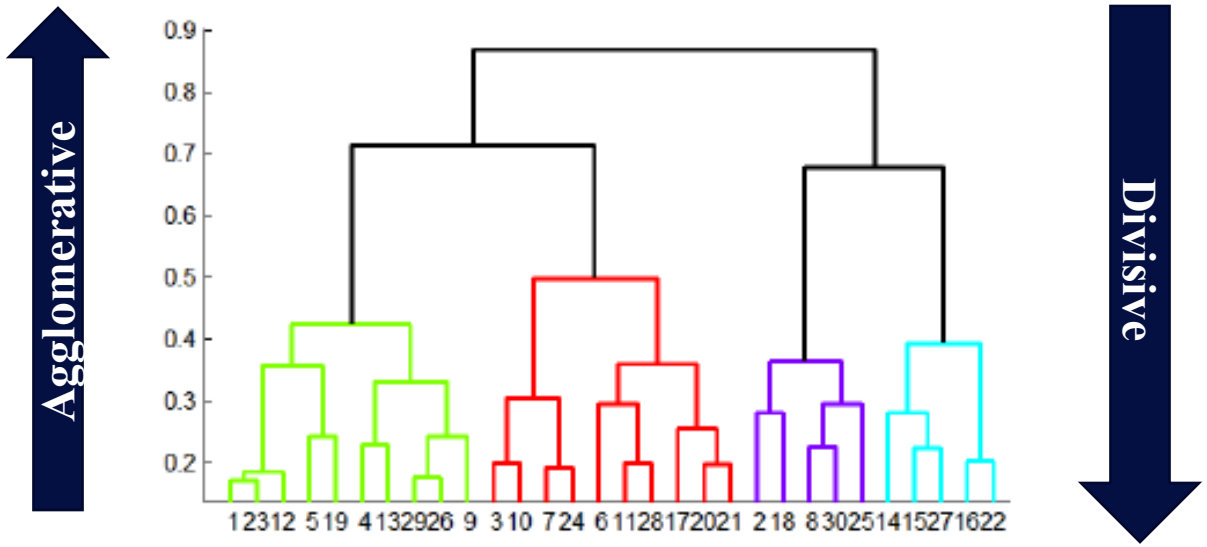
- Euclide distance
- Jaccard distance
- Cosine distance



Jaccard	**0.666667**
Euclide	**10.72381**
Cosine	**0.024447**

Hierarchical Clustering

- Use distance matrix as clustering criteria. This method does not require the number of clusters ***k*** as an input, but needs a **termination condition**



Key operation: Repeatedly combine two nearest clusters

Three important questions:

1) How do you represent a cluster of more than one point?

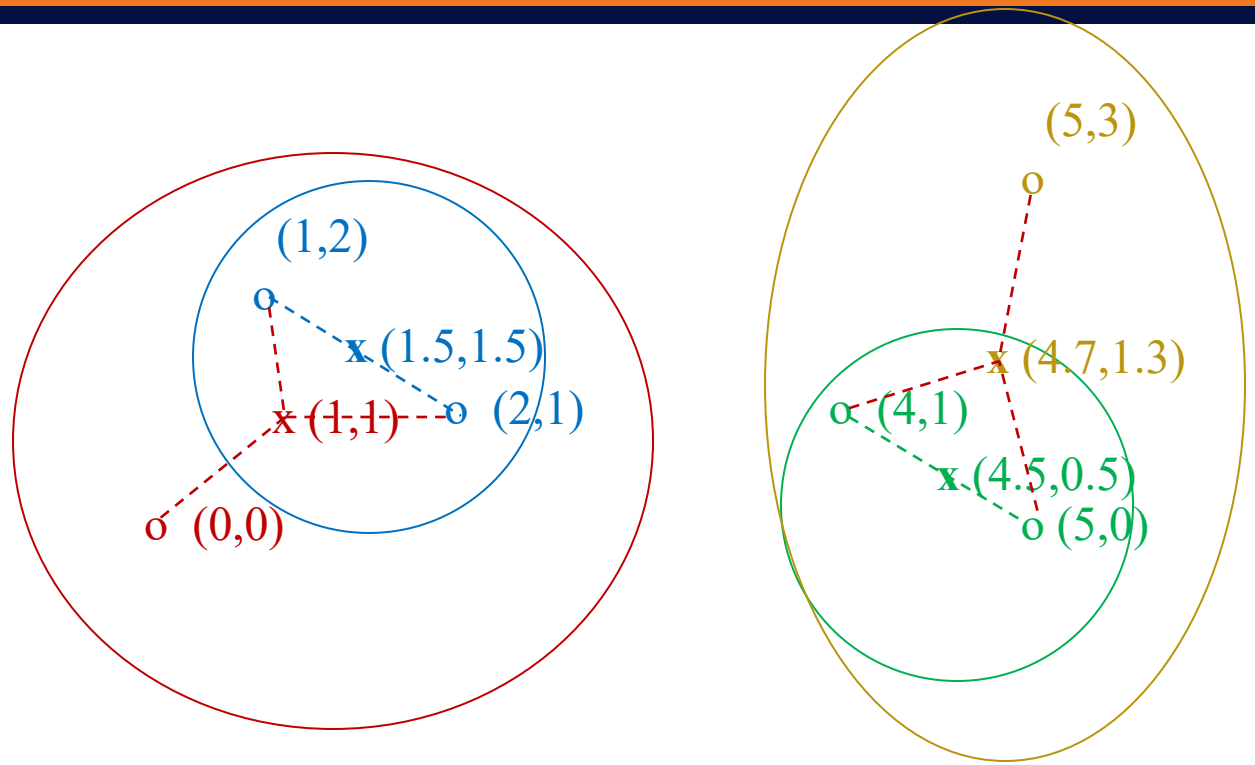
2) How do you determine the "nearness" of clusters?

3) When to stop combining clusters?

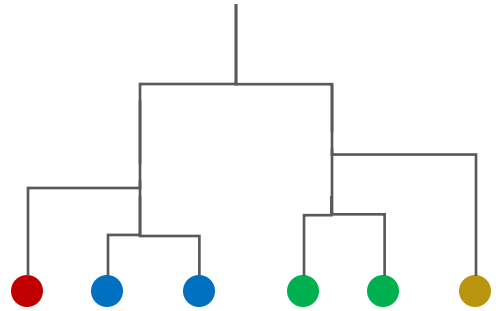
Hierarchical Clustering

- **How to represent a cluster of many points?**
  - **Key problem:** As you merge clusters, how do you represent the "location" of each cluster, to tell which pair of clusters is closest?
  - **Euclidean case:** each cluster has a centroid = average of its (data)points
- **How to determine "nearness" of clusters?**
  - Measure cluster distances by distances of centroids

Example: Hierarchical clustering



Data:
o … data point
x … centroid



Dendrogram

Hierarchical Clustering

What about the non-metric space?

- The only "locations" we can talk about are the points themselves
  - i.e., there is no "average" of two points
- **Approach 1:**
  1. **How to represent a cluster of many points?**

     clustroid (medoid) = (data)point "closest" to other points
  2. **How do you determine the "nearness" of clusters?**

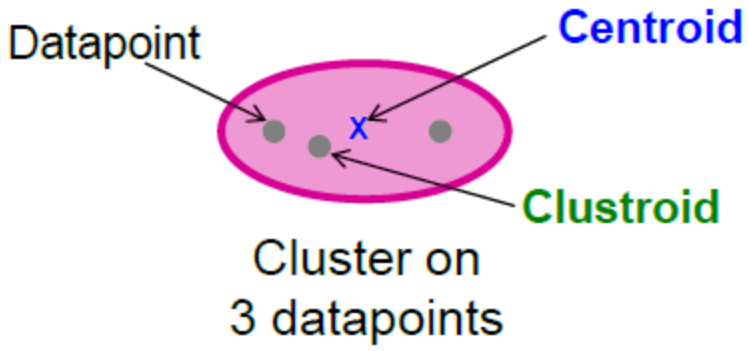
     Treat clustroid (medoid) as if it were centroid, when computing inter-cluster distances

Closest point

(1) How to represent a cluster of many points?

clustroid = point "closest" to other points

- **Possible meanings of "closest":**
  - Smallest maximum distance to other points
  - Smallest average distance to other points
  - Smallest sum of squares of distances to other points



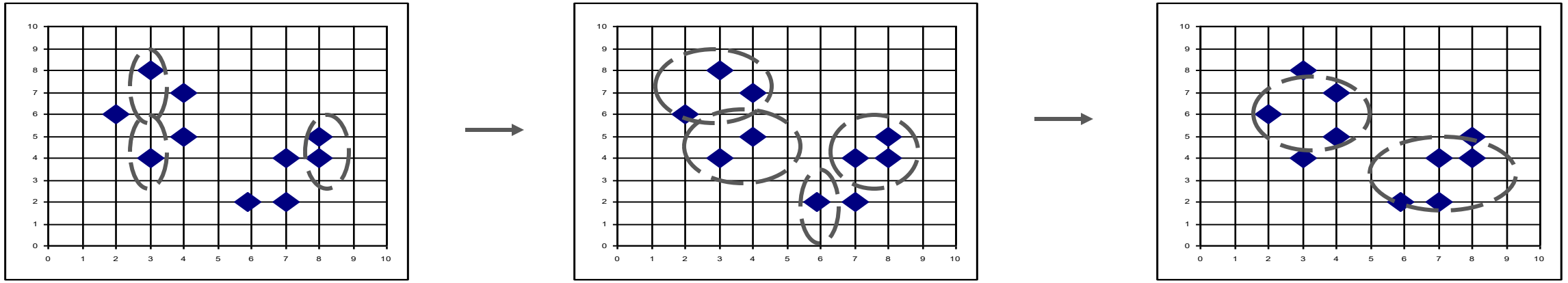
Nearness of cluster

(2) How do you determine the "nearness" of clusters?

- **Approach 2:**
  - Intercluster distance = minimum of the distances between any two points, one from each cluster
- **Approach 3:**
  - Pick a notion of "cohesion" of clusters, e.g., maximum distance from the clustroid
    - Merge clusters whose **union** is most **cohesive**
  - **Approach 3.1:** Use the diameter of the merged cluster = maximum distance between points in the cluster
  - **Approach 3.2:** Use the average distance between points in the cluster
  - **Approach 3.3: Use a density-based approach**
    - Take the diameter or avg. distance, e.g., and divide by the number of points in the cluster

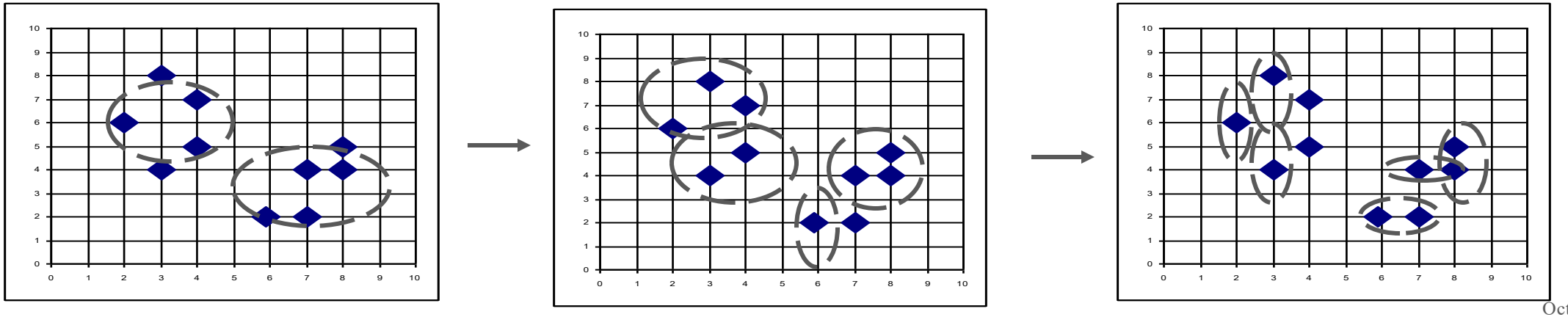
AGNES (Agglomerative Nesting)

- Introduced in Kaufmann and Rousseeuw (1990)
- Use the Single-Link method and the dissimilarity matrix.
- Merge nodes that have the least dissimilarity
- Go on in a non-descending fashion
- Eventually all nodes belong to the same cluster



DIANA (Divisive Analysis)

- Introduced in Kaufmann and Rousseeuw (1990)
- Inverse order of AGNES
- Eventually each node forms a cluster on its own



Implementation

- **Naïve implementation of hierarchical clustering:**
  - At each step, compute pairwise distances between all pairs of clusters, then merge
  - $O(N^3)$
- Careful implementation using priority queue can reduce time to $O(N^2 \log N)$
  - **Still too expensive for really big datasets that do not fit in memory**

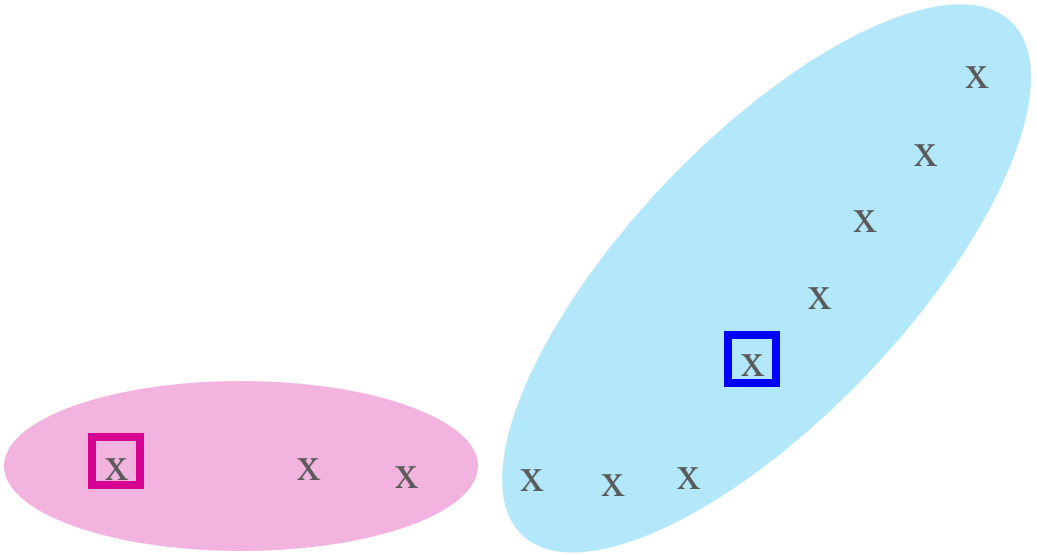
Partitioning Algorithms: Basic Concept

- Partitioning method: Construct a partition of a database ***D*** of ***n*** objects into a set of ***k*** clusters
- Given a *k*, find a partition of *k clusters* that optimizes the chosen partitioning criterion
  - Global optimal: exhaustively enumerate all partitions
  - Heuristic methods: *k-means* and *k-medoids* algorithms
- *k-means* (MacQueen'67): Each cluster is represented by the center of the cluster
- *CURE* **:** Extension of k-means to clusters of arbitrary shapes
- *BFR* (Bradley , Fayyad , Reina '98): a variant of *k-means designed to handle* ***very large (disk-resident) data sets***

The *K-Means* Clustering Method

- Assumes Euclidean space/distance
- Start by picking ***k,*** the number of clusters
- Initialize clusters by picking one point per cluster (called centroid of cluster)
  - Example: Pick one point at random, then *k-1 other points, each as far away as possible from the previous points*
- The ***k-means*** algorithm is implemented in the following steps:
  1. For each point, place it in the cluster whose current centroid it is nearest
  2. After all points are assigned, update the locations of centroids of the *k clusters*
  3. Reassign all points to their closest centroid
     - Sometimes moves points between clusters
  - ❖ Repeat 2 and 3 until convergence.
    - ➢ Convergence: Points don't move between clusters and centroids stabilize

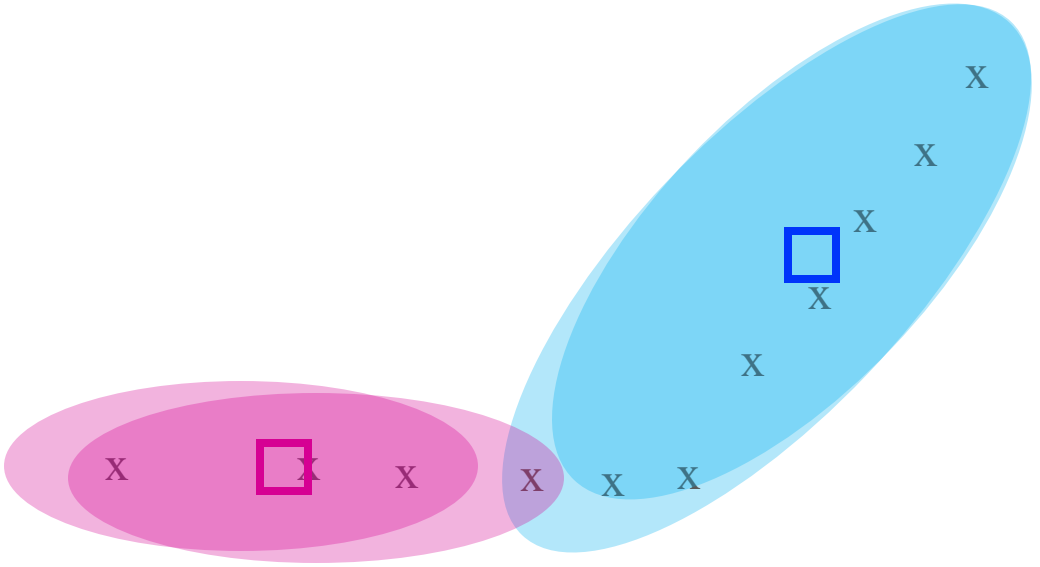
Example: Assigning Clusters



x … data point
□ … centroid

Clusters after round 1

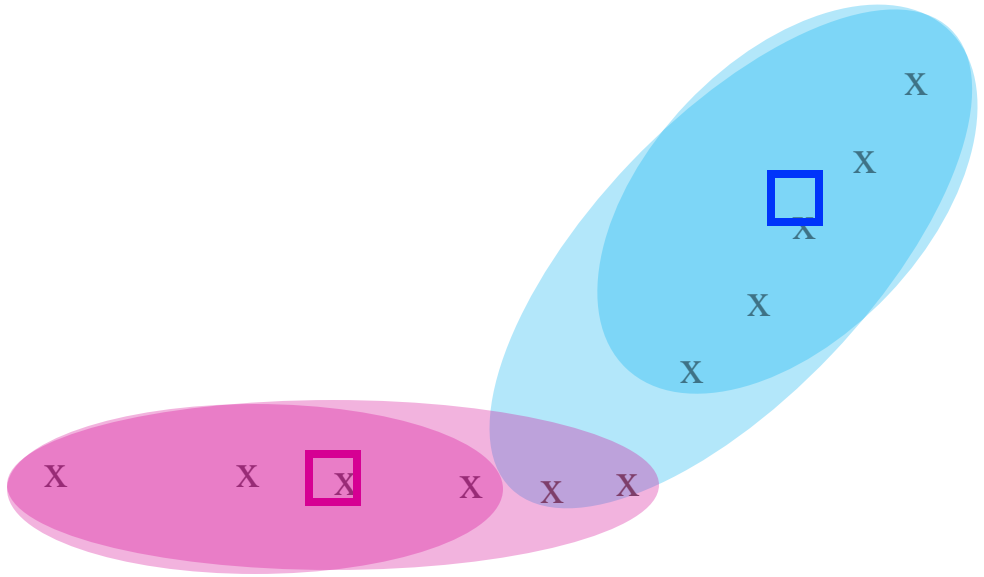
Example: Assigning Clusters



x … data point
□ … centroid

Clusters after round 2

Example: Assigning Clusters



x … data point
□ … centroid

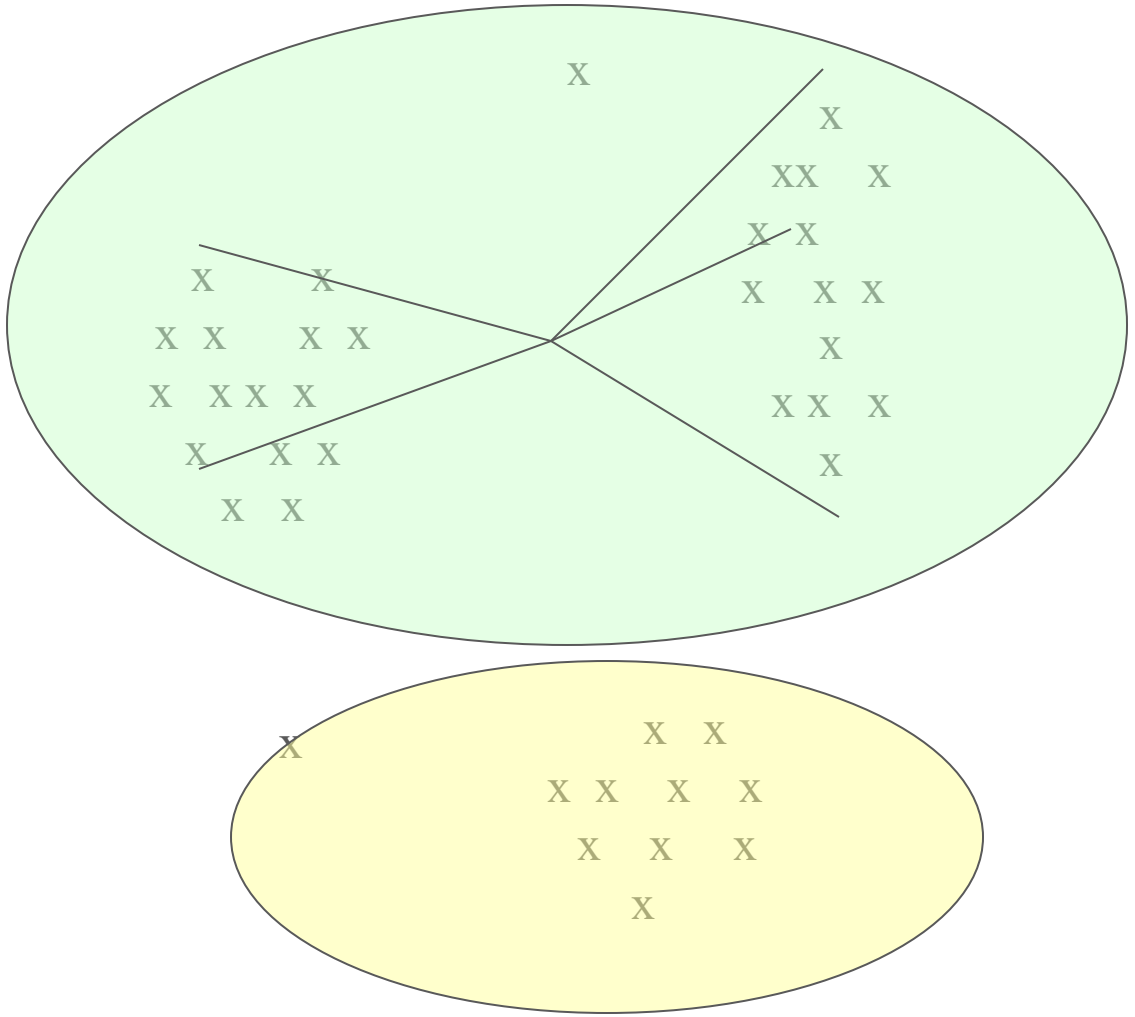
Clusters at the end

Comments on the *K-Means* Method

- Strength
  - *Relatively efficient*: $O(tkn)$, where n is # objects, k is # clusters, and t is # iterations. Normally, $k, t << n$.
  - Often terminates at a *local optimum*.
- Weakness
  - Applicable only when *mean* is defined, then what about categorical data?
  - Need to specify *k,* the *number* of clusters, in advance
  - Unable to handle noisy data and *outliers*
  - Not suitable to discover clusters with *non-convex shapes*

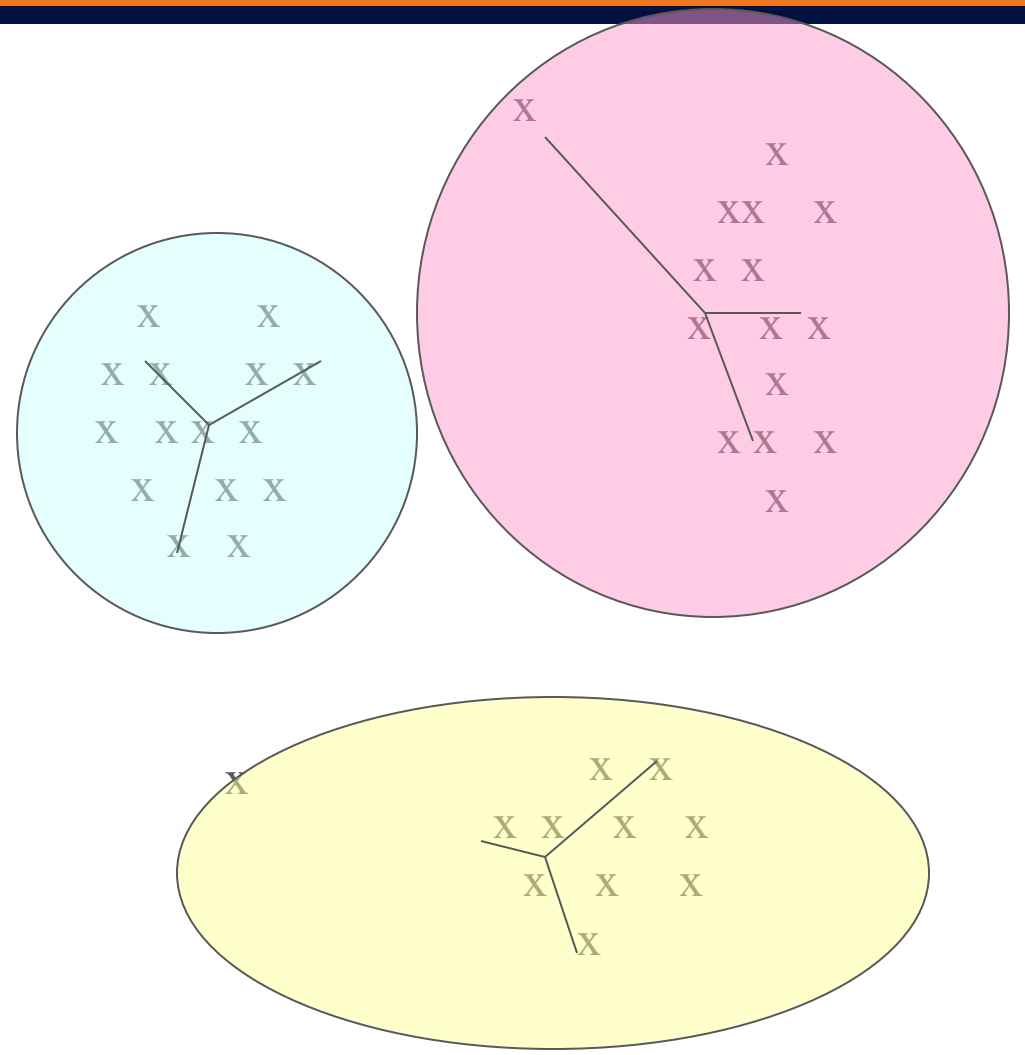
Example: Picking *k*

Too few;
many long
distances
to centroid.



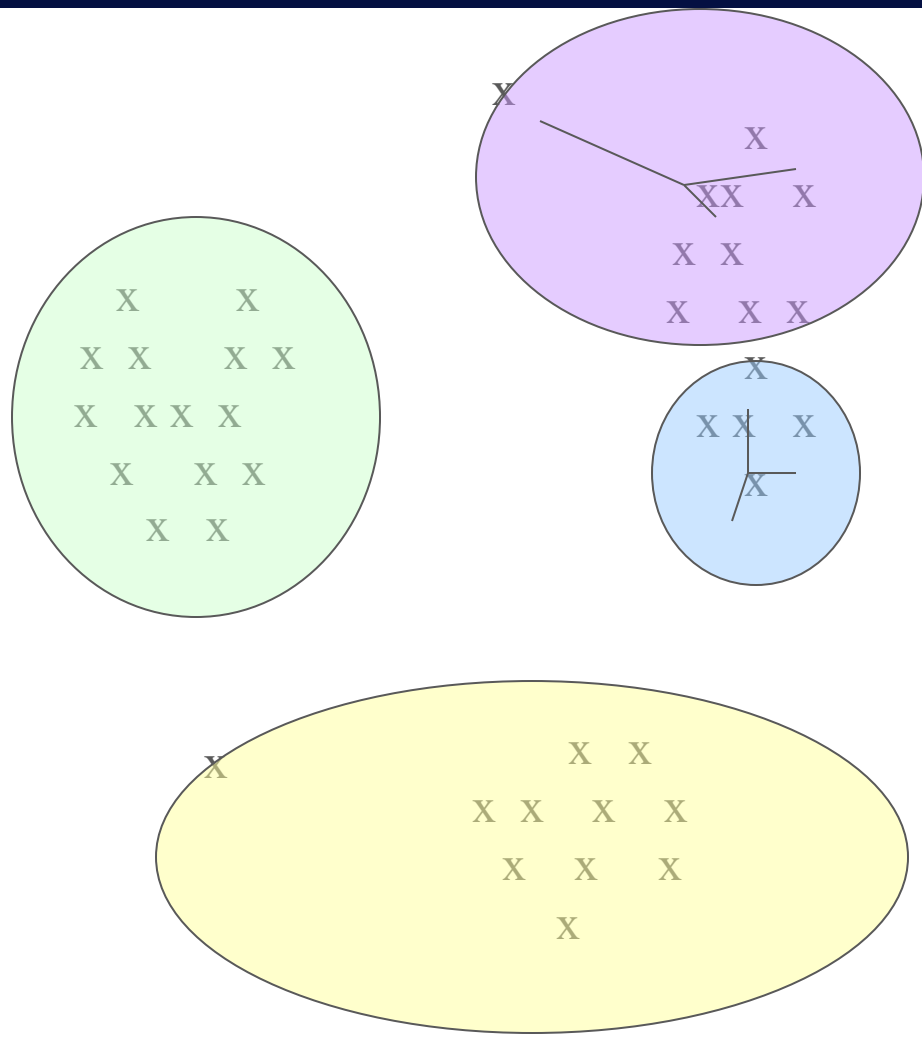
Example: Picking *k*

Just right;
distances
rather short.



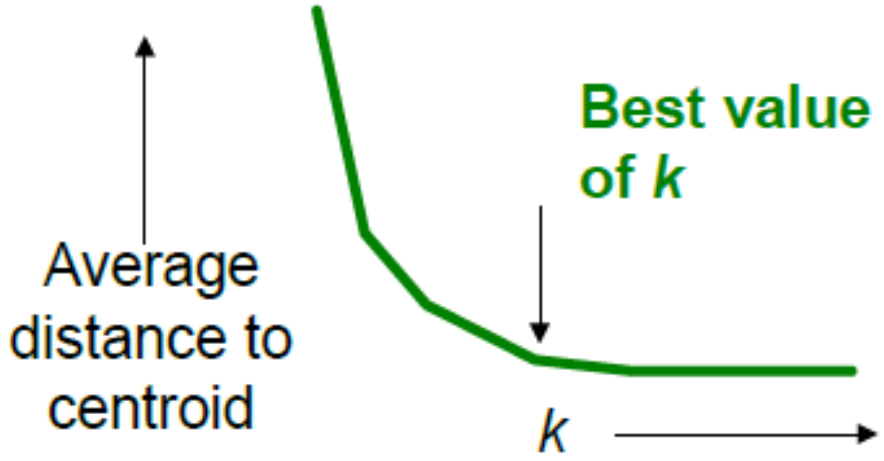
Example: Picking *k*

Too many;
little improvement
in average
distance.



How to pick *k*?

- Try different **k,** looking at the change in the average distance to centroid as k increases
- Average falls rapidly until right **k, then changes little**



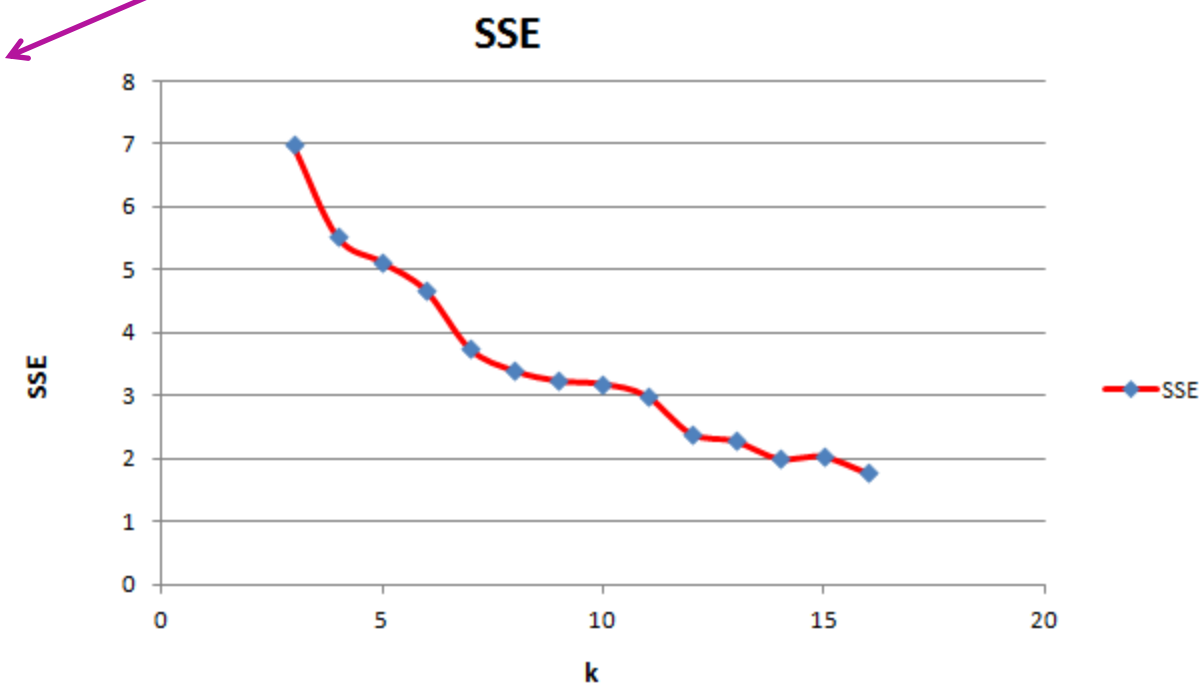
How to pick *k*?

6.2	5.6417
2.8667	2.7833
4.5333	4.1
1.3778	1.25

Clustered Instances,k =16

0	9 (6%)
1	12 (8%)
2	16 (11%)
3	12 (8%)
4	10 (7%)
5	5 (3%)
6	20 (13%)
7	1 (1%)
8	11 (7%)
9	4 (3%)
10	8 (5%)
11	6 (4%)
12	13 (9%)
13	9 (6%)
14	11 (7%)
15	3 (2%)

Cluster x-y	d(x-y)
0-1	0.8731
1-2	0.7116
2-3	1.7722
3-4	4.2611
4-5	3.5009
5-6	0.9271
6-7	3.1204
7-8	1.2919
8-9	2.5016
9-10	2.6521
10-11	4.2392
11-12	0.6831
12-13	3.6093
13-14	5.4817
14-15	1.5764



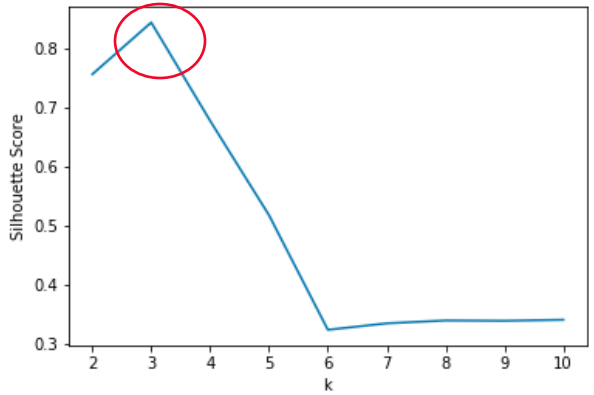
6.2375	5.6
2.4375	2.6667
5.0375	4.95
1.5625	1.9833

Elbow method

Silhouette method

- A value rangs from -1 to 1. If this value close to 1, it indicate the point placed in the correct cluster.
- The silhouette value s(i) for each point i :

$$s(i) = \frac{b\{i) - a(i)}{\max\{a(i), b(i)\}}, if\ |C_i| > 1 \qquad s(i) = 0\ if\ |C_i| = 0$$



- *a(i)*: the distance of the point i to its own cluster
- *b(i)*: the dissimilarity of the point i from points in other clusters.

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \# j} d(i,j) \qquad b(i) = min_{i \# j} \frac{1}{|C_j|} \sum_{j \in C_j} d(i,j)$$

C_j: the closest cluster to the cluster of i

- The average score for every point i

$$score = mean\{s(i)\}$$

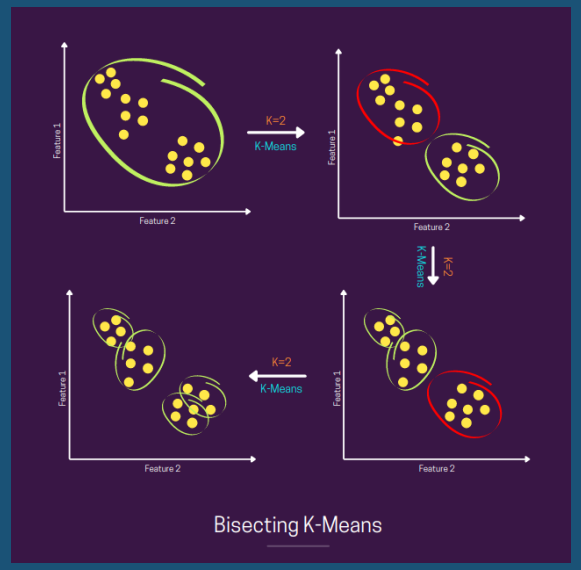
Bisecting Kmeans

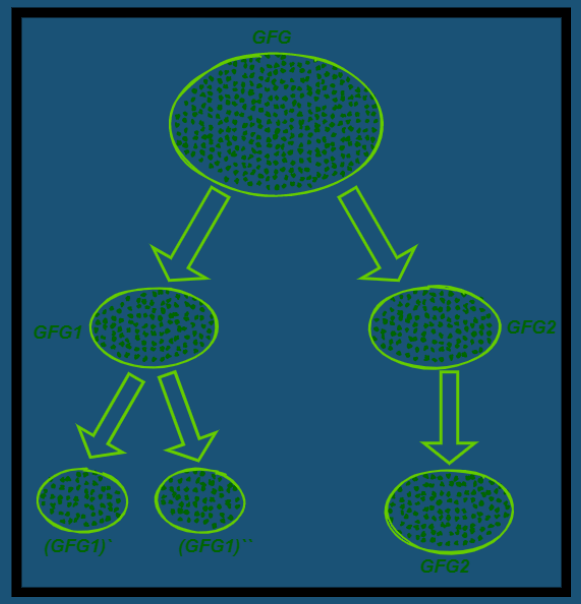
- Hybrid of divisive clustering and Kmeans
- Split one cluster into 2 sub clusters in each iteration
- Bisecting Kmeans Algorithm with k*:
  - Initialization: one big cluster of all data points
  - Step 1: Use Kmeans (k=2) to split this cluster into 2 sub clusters.
  - Step 2: Calculate SSD of these 2 sub clusters, select the one that has the largest distance and return to step 1.
  - Repeat Step 1 and Step 2 until the number of leaf cluster = k*.

BISECTING K-MEANS

Hierarchical Bisecting Process:

Step 1: Start with all data in one cluster



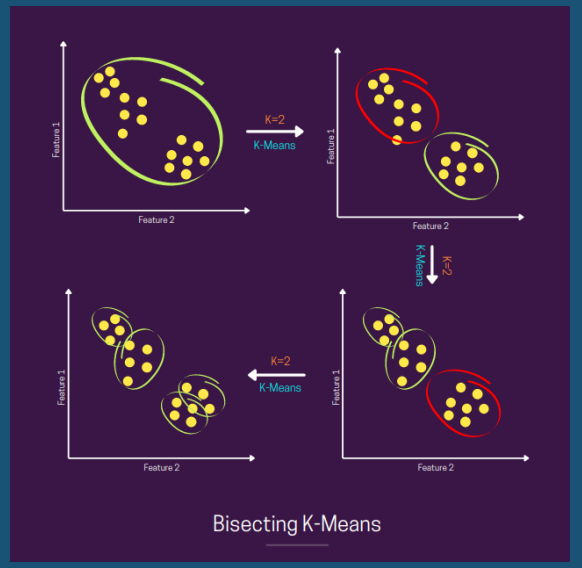


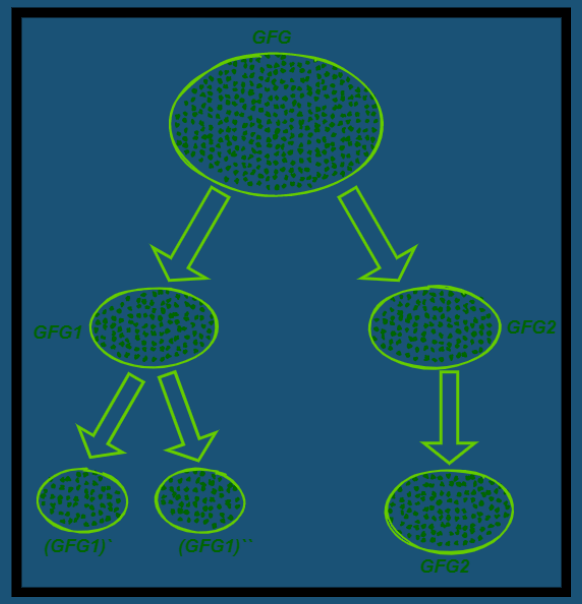
Step 2: Select a cluster to split

• Choose the largest cluster or the one with highest SSE (Sum of Squared Errors)

Step 3: Apply K-means (k=2) to split the selected cluster

• Run standard K-means with k=2 on the selected cluster



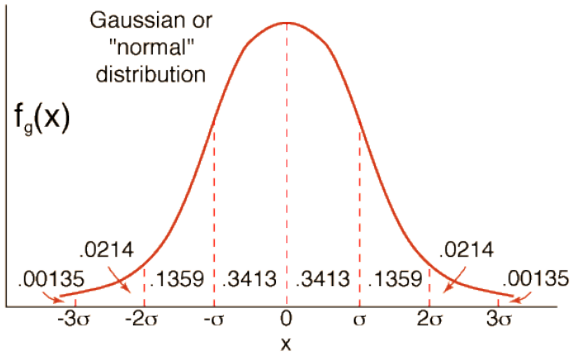


Step 4: Repeat until k clusters are formed

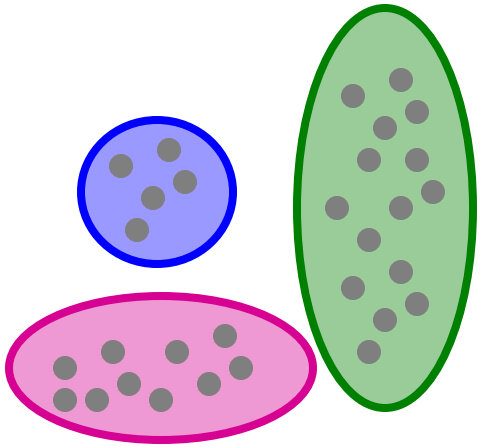
• Continue selecting and splitting clusters until the desired number of clusters is reached

BFR Algorithm

- **BFR** [Bradley-Fayyad-Reina] is a variant of *k*-means designed to handle **very large** (disk-resident) data sets



- **Assumes** that clusters are normally distributed around a centroid in a Euclidean space
  - Standard deviations in different dimensions may vary
    - Clusters are axis-aligned ellipses



- **Efficient way to summarize clusters** (want memory required O(clusters) and not O(data))

BFR Algorithm

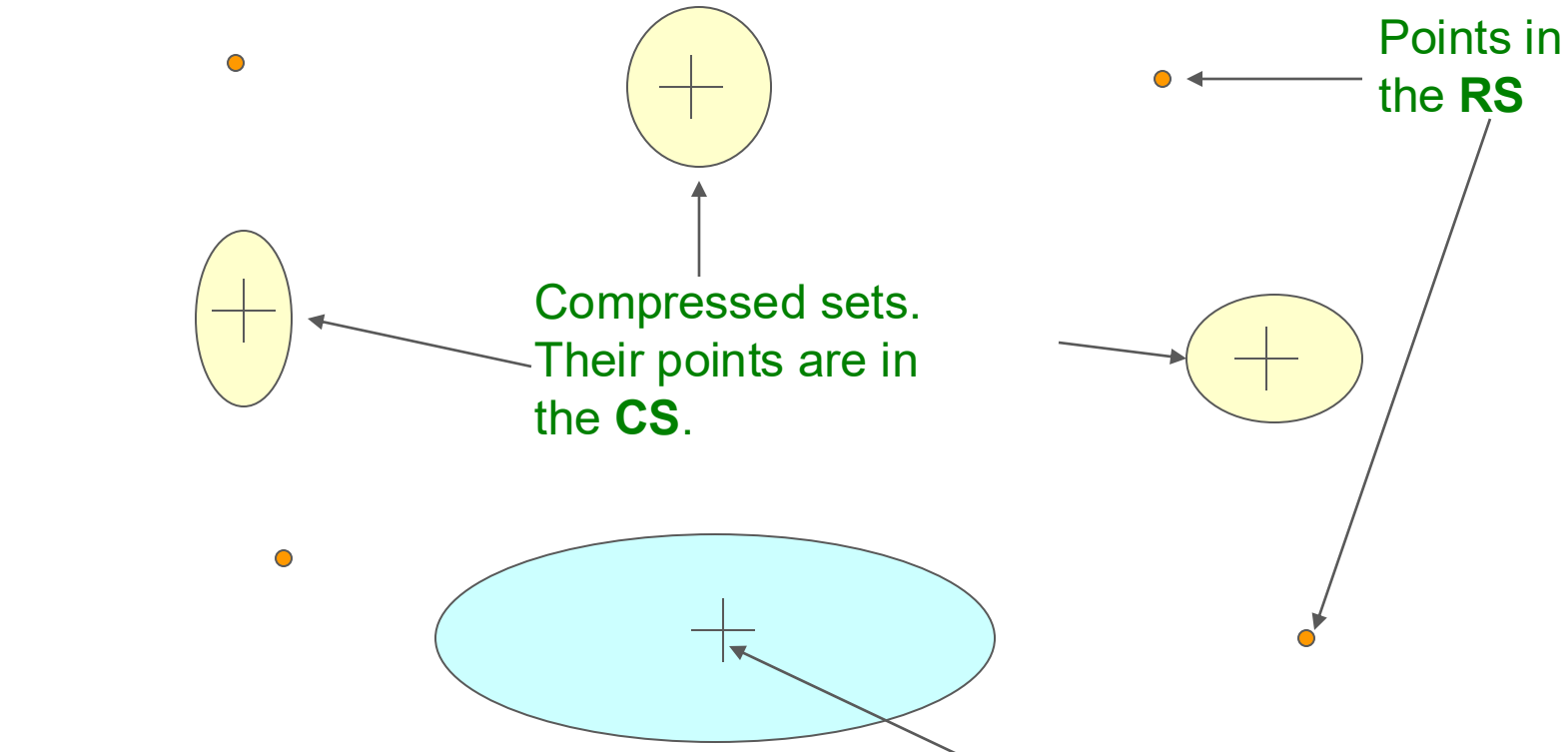
- Points are read from disk one main-memory-full at a time
- **Most points from previous memory loads are summarized by simple statistics**
- To begin, from the initial load we select the initial ***k*** centroids by some sensible approach:
  - Take ***k*** random points
  - Take a small random sample and cluster optimally
  - Take a sample; pick a random point, and then ***k–1*** more points, each as far from the previously selected points as possible

Three Classes of Points

3 sets of points which we keep track of:

- **Discard set (DS):**
  - Points close enough to a centroid to be summarized
- **Compression set (CS):**
  - Groups of points that are close together but not close to any existing centroid
  - These points are summarized, but not assigned to a cluster
- **Retained set (RS):**
  - Isolated points waiting to be assigned to a compression set

BFR: "Galaxies" Picture

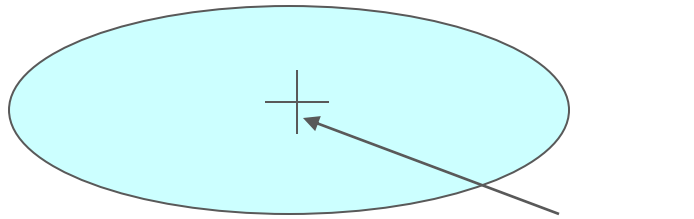


Discard set (DS): Close enough to a centroid to be summarized
Compression set (CS): Summarized, but not assigned to a cluster
Retained set (RS): Isolated points

Summarizing Sets of Points

For each cluster, the discard set (DS) is summarized by:

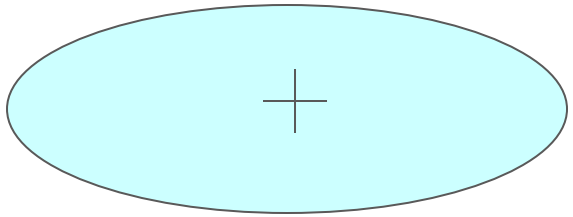
- The number of points, ***N***
- The vector ***SUM***, whose i^{th} component is the sum of the coordinates of the points in the i^{th} dimension
- The vector ***SUMSQ***: i^{th} component = sum of squares of coordinates in i^{th} dimension



Summarizing Points: Comments

- $2d + 1$ values represent any size cluster
  - d = number of dimensions
- Average in **each dimension** (**the centroid**) can be calculated as $\mathbf{SUM}_i / N$
  - $\mathbf{SUM}_i$ = i^{th} component of SUM
- Variance of a cluster's discard set in dimension i is: $(\mathbf{SUMSQ}_i / N) - (\mathbf{SUM}_i / N)^2$
  - And standard deviation is the square root of that
- **Next step: Actual clustering**

Note: Dropping the "axis-aligned" clusters assumption would require storing full covariance matrix to summarize the cluster. So, instead of **SUMSQ** being a ***d***-dim vector, it would be a ***d x d*** matrix, which is too big!



The "Memory-Load" of Points

Processing the "Memory-Load" of points (1):

- **1)** Find those points that are "**sufficiently close**" to a cluster centroid and add those points to that cluster and the **DS**
  - These points are so close to the centroid that they can be summarized and then discarded
- **2)** Use any main-memory clustering algorithm to cluster the remaining points and the old **RS**
  - Clusters go to the **CS**; outlying points to the **RS**

Discard set (DS): Close enough to a centroid to be summarized.
Compression set (CS): Summarized, but not assigned to a cluster
Retained set (RS): Isolated points

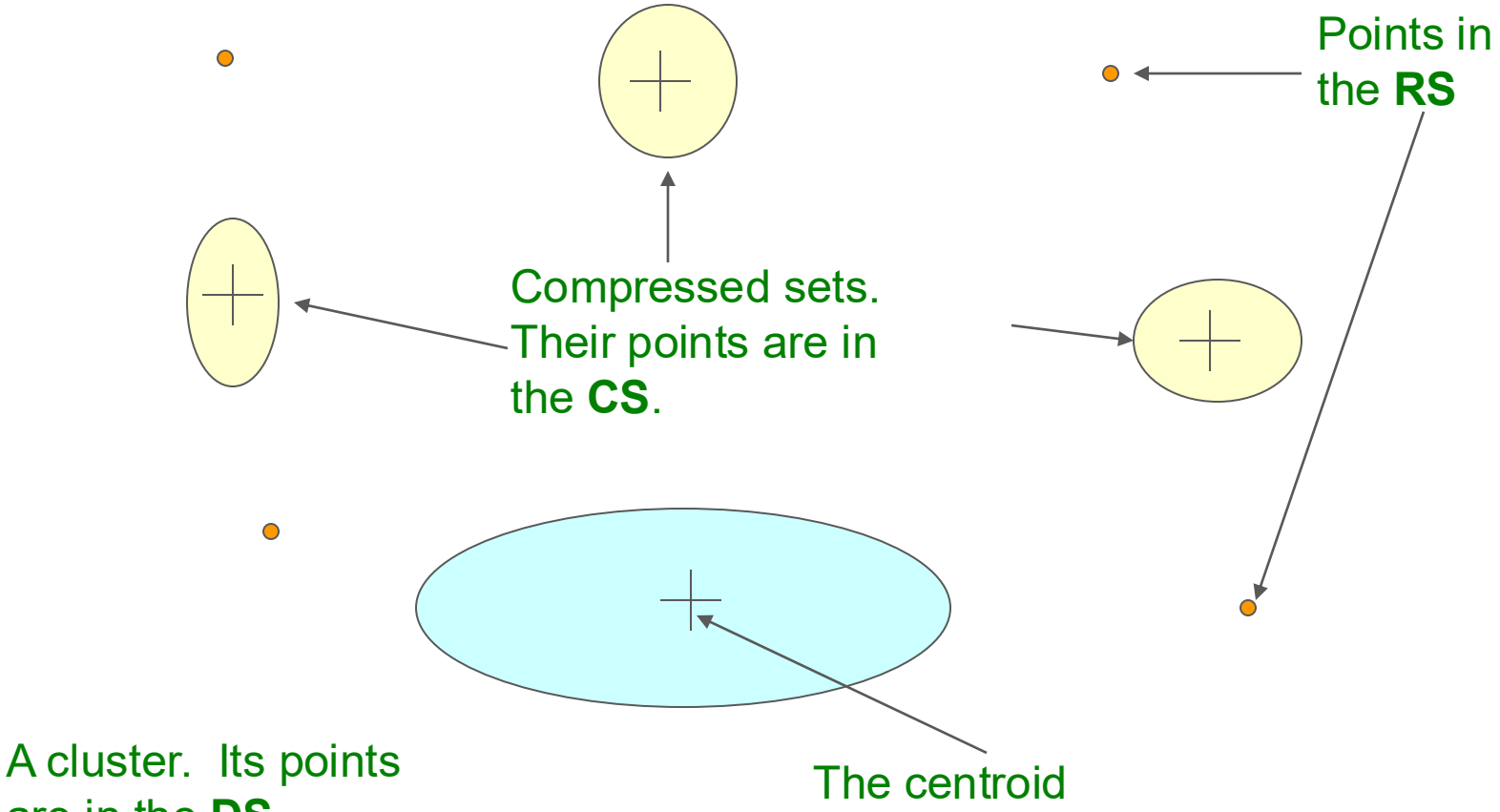
The "Memory-Load" of Points

Processing the "Memory-Load" of points (2):

- **3) DS set:** Adjust statistics of the clusters to account for the new points
  - Add *N*s, ***SUM***s, ***SUMSQ***s
- **4)** Consider merging compressed sets in the **CS**
- **5)** If this is the last round, merge all compressed sets in the **CS** and all **RS** points into their nearest cluster

Discard set (DS): Close enough to a centroid to be summarized.
Compression set (CS): Summarized, but not assigned to a cluster
Retained set (RS): Isolated points

BFR: "Galaxies" Picture



Discard set (DS): Close enough to a centroid to be summarized
Compression set (CS): Summarized, but not assigned to a cluster
Retained set (RS): Isolated points

A Few Details…

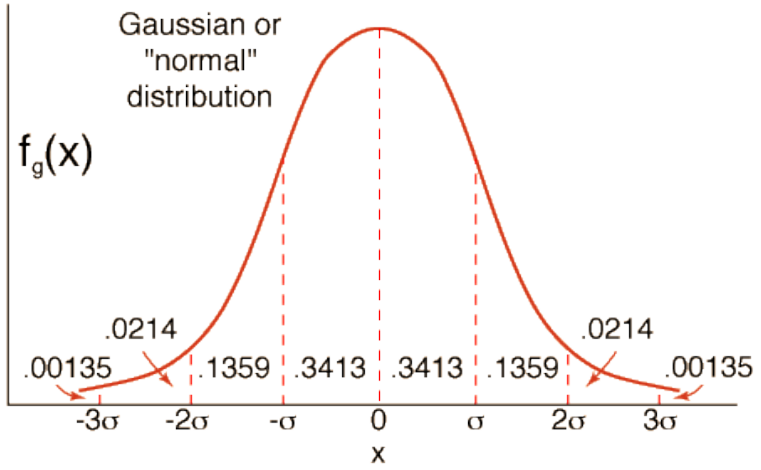
- **Q1) How do we decide if a point is "close enough" to a cluster that we will add the point to that cluster?**

- **Q2) How do we decide whether two compressed sets (CS) deserve to be combined into one?**

How Close is Close Enough?

- **Q1) We need a way to decide whether to put a new point into a cluster (and discard)**

- **BFR suggests two ways:**
  - The **Mahalanobis distance** is less than a threshold
  - **High likelihood of the point belonging to currently nearest centroid**



Mahalanobis Distance

- **Normalized Euclidean distance from centroid**

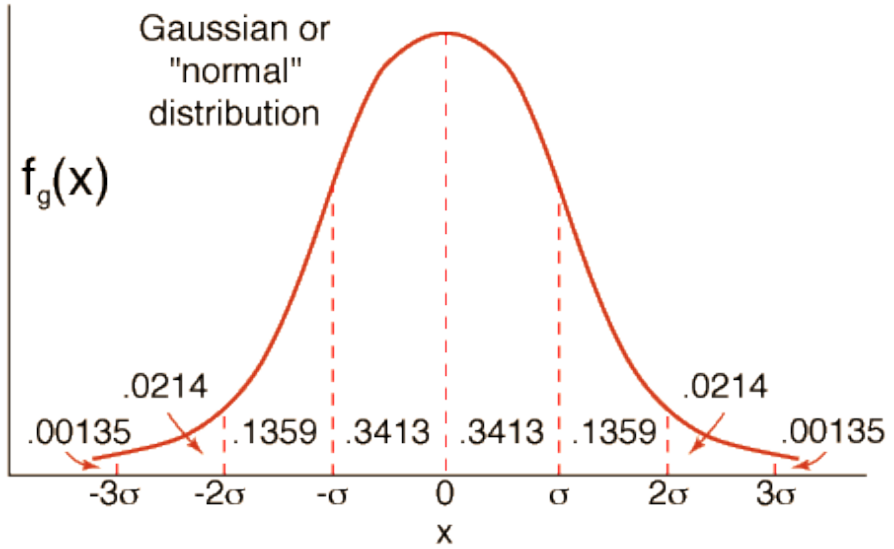
- For point $(x_1, \ldots, x_d)$ and centroid $(c_1, \ldots, c_d)$
  1. Normalize in each dimension: $y_i = (x_i - c_i) / \sigma_i$
  2. Take sum of the squares of the y_i
  3. Take the square root

$$d(x, c) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - c_i}{\sigma_i}\right)^2} = \sqrt{\sum_{i=1}^{d} y_i^2}$$

σ_i … standard deviation of points in the cluster in the i^{th} dimension

Mahalanobis Distance

- Suppose point x is one standard deviation away from centroid in each dimension,
  - y_i =1
  - The MD of x: d(x,C) = $\sqrt{\boldsymbol{d}}$
- 68% of points has MD ≤ $\sqrt{\boldsymbol{d}}$
- 95% of points has MD ≤ $2\sqrt{\boldsymbol{d}}$
- 98% of points has MD ≤ $3\sqrt{\boldsymbol{d}}$

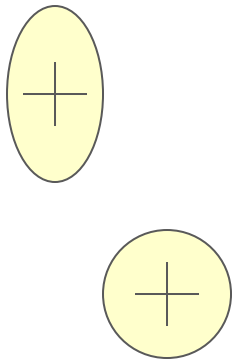


Accept point x into cluster C if its MD from cluster centroid is less than a threshold (e.g $3\sqrt{\boldsymbol{d}}$)

Should 2 CS clusters be combined?

Q2) Should 2 CS subclusters be combined?

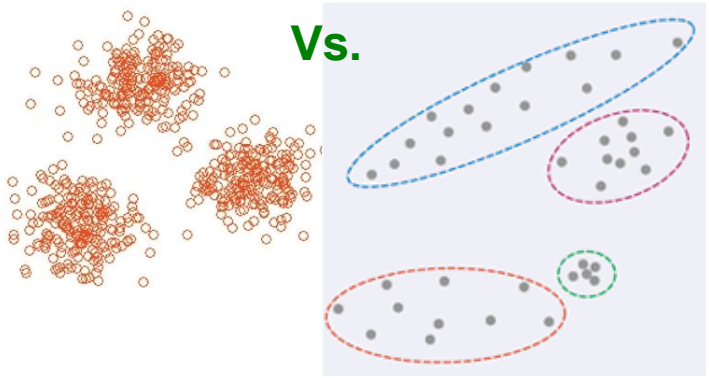
- Compute the variance of the combined subcluster
  - ***N***, ***SUM***, and ***SUMSQ*** allow us to make that calculation quickly
- Combine if the combined variance is below some threshold



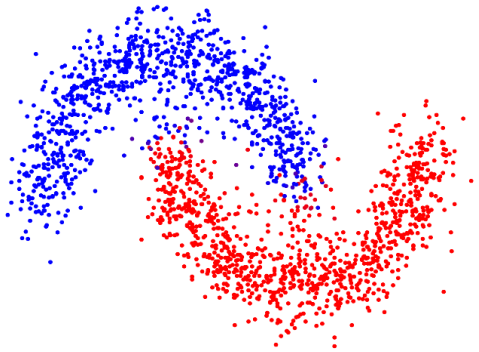
- **Many alternatives:** Treat dimensions differently, consider density

The CURE Algorithm

- **Problem with BFR/*k*-means:**
  - Assumes clusters are normally distributed in each dimension
  - And axes are fixed – ellipses at an angle are ***not OK***



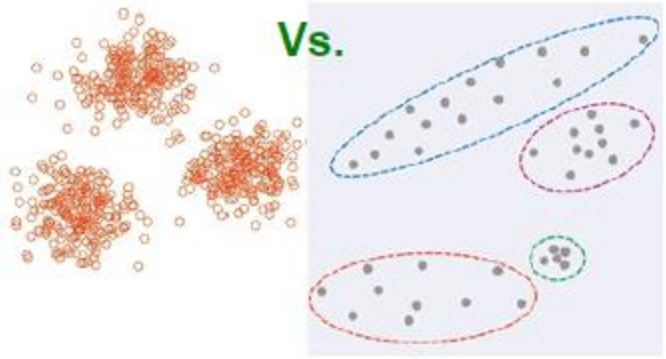
- **CURE (Clustering Using REpresentatives):**
  - Assumes a Euclidean distance
  - Allows clusters to assume any shape
  - **Uses a collection of representative points to represent clusters**

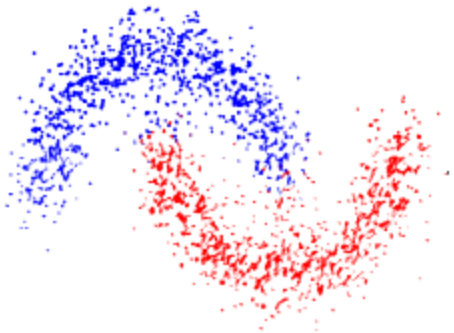


The Cure algorithm step

2 Pass algorithm. Pass 1:

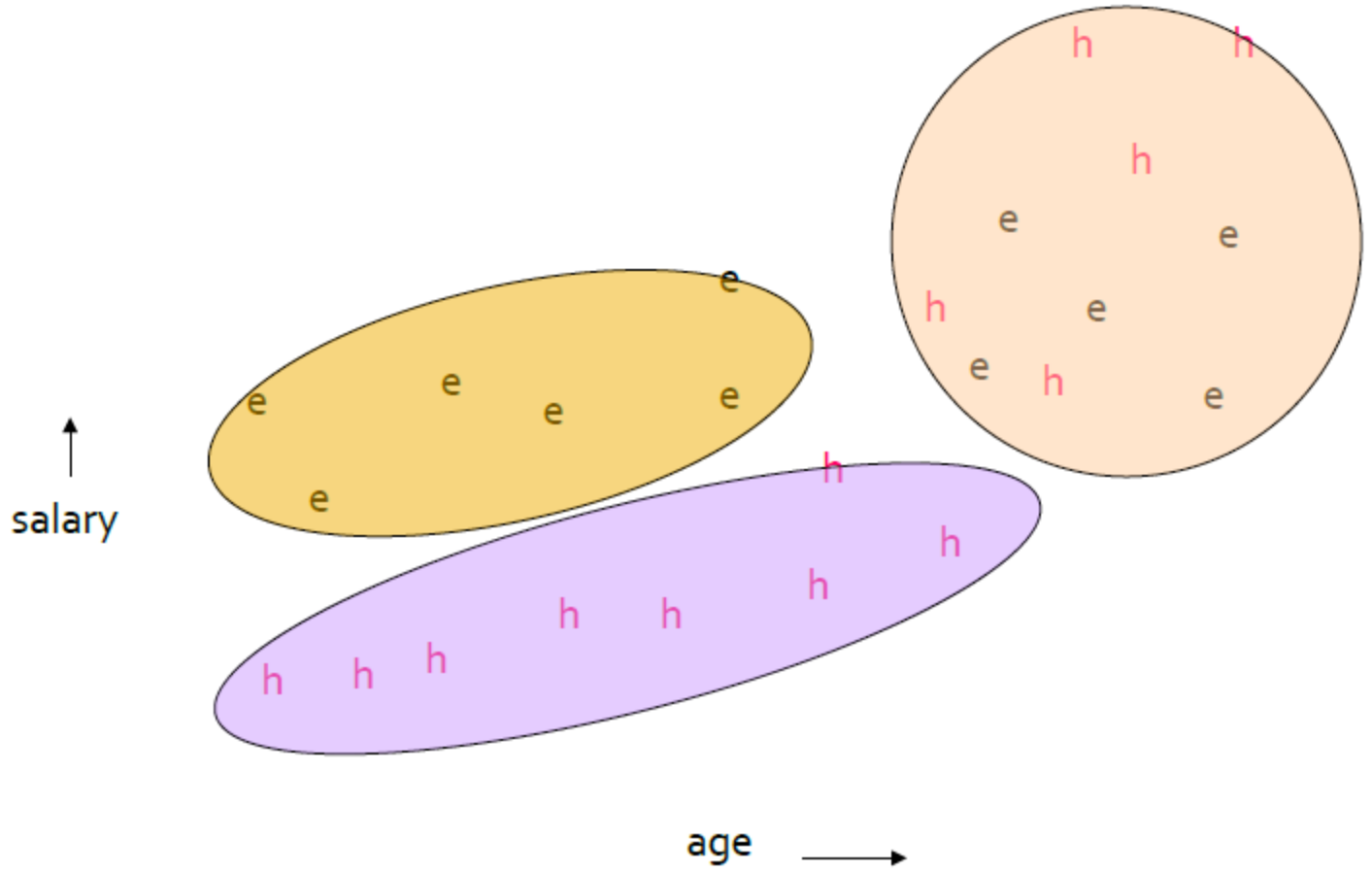
1. **Pick a random sample of points that fit in main memory**
2. **Initial clusters:**
   - Cluster these points hierarchically – group nearest points/clusters
3. **Pick representative points:**
   - For each cluster, pick a sample of points, as dispersed as possible
   - From the sample, pick representatives by moving them (say) 20% toward the centroid of the cluster



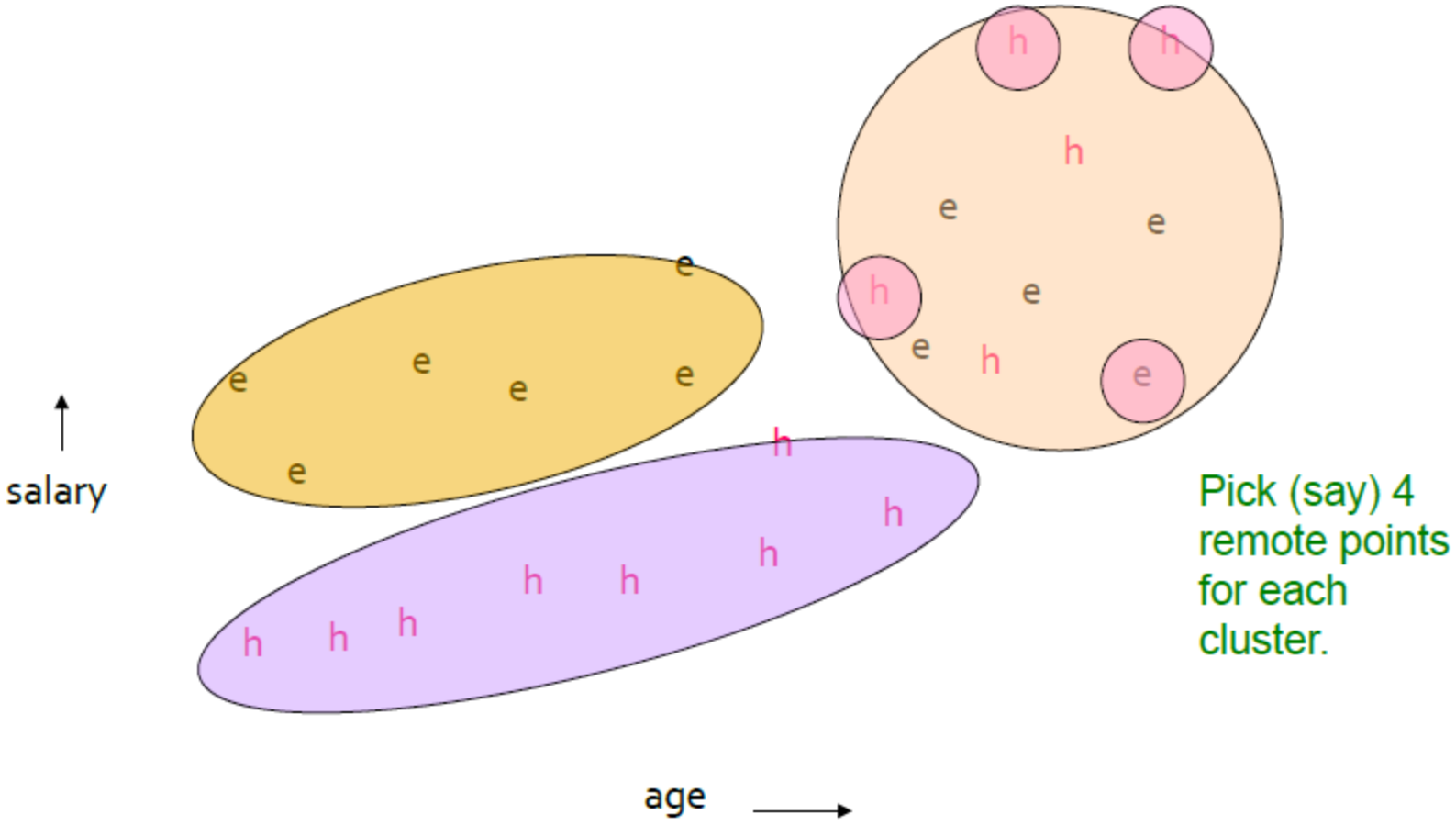


The Cure algorithm step: example

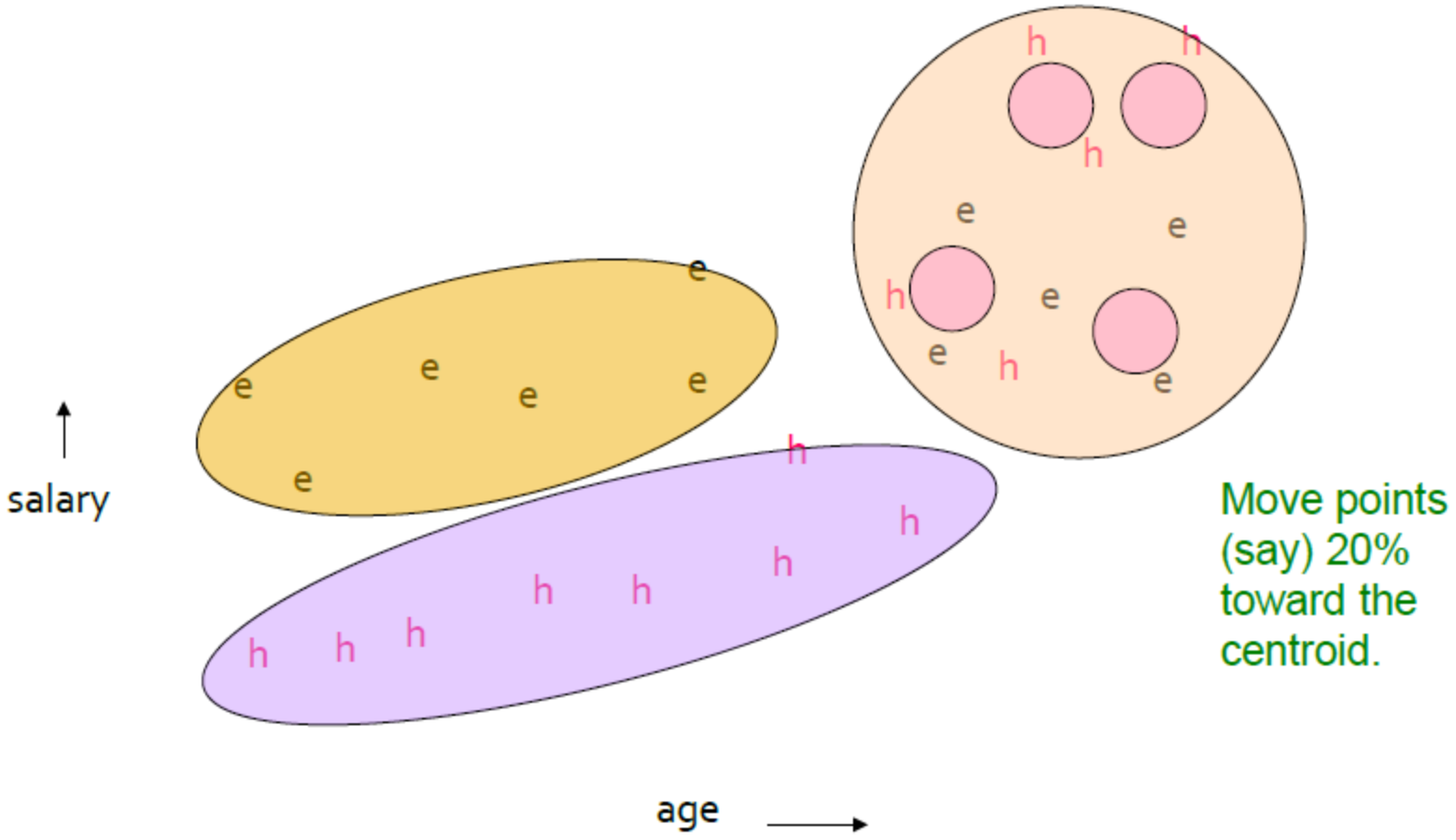
Initial clusters



Pick dispersed points



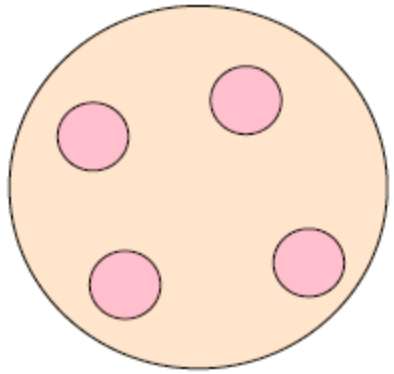
Pick dispersed points



The Cure algorithm step

Pass 2:

- Now, rescan the whole dataset and visit each point ***p*** in the data set
- Place it in the "closest cluster"
  - Normal definition of "closest": Find the closest representative to p and assign it to representative's cluster



p

Distributed Clustering methods

MapReduce-KMeans

Working Principle:

- Distributes data across multiple computing nodes
- Map: Each node assigns data points to nearest cluster
- Shuffle: Groups points by assigned cluster
- Reduce: Recalculates new cluster centroids
- Iterates until convergence

Advantages and Disadvantages:

✓ Linear scalability with data size

✓ Efficient processing of large data on clusters

✗ Ineffective with complex-shaped clusters

✗ Requires predefined number of clusters k

DBDC (Density Based Distributed Clustering)

Working Principle:

- Partitions data into local subsets
- Applies DBSCAN on each local partition
- Extracts representatives from each local cluster
- Clusters representatives at global level
- Combines local and global clustering results

Advantages and Disadvantages:

✓ Detects clusters of arbitrary shapes

✓ No need to predefine number of clusters

✗ Sensitive to density and distance parameters

✗ Difficult to handle varying density data

Deep clustering network

Working Principle:

- Combines autoencoder with K-means
- Learns feature representation through autoencode
- Simultaneously optimizes reconstruction and clustering
- Uses combined loss function
- Alternating optimization between network and clusters

Combined Loss Function:

$$L = L_r + \lambda \cdot L_c$$

$$L_r = \|X - X'\|^2_F \quad \text{(Reconstruction loss)}$$

$$L_c = \|Z - C \cdot S^T\|^2_F \quad \text{(Clustering loss)}$$

λ: Balance parameter between loss components

Parameter Explanation:

X and X':

- X: Input data
- X': Data reconstructed by autoencoder

Z: Latent representation matrix

- $Z \in \mathbb{R}^{(n \times d)}$: n is number of data points, d is latent dimension
- Z_i: Latent representation of data point i

C: Cluster centroid matrix

- $C \in \mathbb{R}^{(k \times d)}$: k is number of clusters, d is latent dimension
- C_j: Centroid of cluster j in latent space

S: Cluster assignment matrix

- $S \in \{0,1\}^{(n \times k)}$: Binary matrix
- $S_{ij} = 1$ if point i belongs to cluster j, otherwise $S_{ij} = 0$
- $\sum_j S_{ij} = 1$: Each point belongs to exactly one cluster

Recall: similarity/ dissimilarity measure

Normalization

- **Min-max normalization**: to [new_min$_A$, new_max$_A$]

$$v_i' = \frac{v_i - min_A}{max_A - min_A}(new\_max_A - new\_min_A) + new\_min_A.$$

  - Ex. Let income range \$12,000 to \$98,000 normalized to [0.0, 1.0]. Then \$73,000 is mapped to $\frac{73{,}600 - 12{,}000}{98{,}000 - 12{,}000}(1.0 - 0) + 0 = 0.716.$

- **Z-score normalization** (μ: mean, σ: standard deviation):

$$v_i' = \frac{v_i - \bar{A}}{\sigma_A},$$

  - Ex. Let μ = 54,000, σ = 16,000. Then $\frac{73{,}600 - 54{,}000}{16{,}000} = 1.225.$

- **Normalization by decimal scaling** $v_i' = \frac{v_i}{10^j},$

Where j is the smallest integer such that Max(|ν'|) < 1

Interval-valued variables

- Standardize data

✓Calculate the mean absolute deviation:

$$s_f = \frac{1}{n}(|x_{1f} - m_f| + |x_{2f} - m_f| + \cdots + |x_{nf} - m_f|)$$

where $m_f = \frac{1}{n}(x_{1f} + x_{2f} + \ldots + x_{nf})$.

✓Calculate the standardized measurement (*z-score*)

$$z_{if} = \frac{x_{if} - m_f}{s_f}$$

Similarity and Dissimilarity Between Objects

- Distances are normally used to measure the similarity or dissimilarity between two data objects
- Some popular ones include: *Minkowski distance*:

$$d(i,j) = \sqrt{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + |x_{i2} - x_{j2}|^q)}$$

where $i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \ldots, x_{jp})$ are two p-dimensional data objects, and q is a positive integer

- If $q = 1$, d is Manhattan distance

$$d(i,j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \cdots + |x_{ip} - x_{jp}|$$

Similarity and Dissimilarity Between Objects (Cont.)

- *If q = 2, d* is Euclidean distance:

$$d(i,j) = \sqrt{(|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + \ldots + |x_{ip} - x_{jp}|^2 \quad)}$$

  - Properties
    - $d(i,j) \geq 0$
    - $d(i,i) = 0$
    - $d(i,j) = d(j,i)$
    - $d(i,j) \leq d(i,k) + d(k,j)$

Similarity and Dissimilarity Between Objects (Cont.)

- ***Mahalanobis distance:***

$$d(x,c) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - c_i}{\sigma_i}\right)^2}$$

- **Normalized Euclidean distance from centroid**
  - For point $(x_1, \ldots, x_d)$ *and centroid* $(c_1, \ldots, c_d)$
    - Normalize in each dimension: $y_i = (x_i - c_i) / \sigma_i$
    - Take sum of the squares of the y_i
    - Take the square root

Binary Variables

- A contingency table for binary data

		Object *j*		
		1	0	*sum*
Object *i*	1	a	b	$a+b$
	0	c	d	$c+d$
	sum	$a+c$	$b+d$	p

- Simple matching coefficient (if the binary variable is *symmetric*):

$$d(i,j) = \frac{b+c}{a+b+c+d}$$

- Jaccard coefficient (if the binary variable is *asymmetric*):

$$d(i,j) = \frac{b+c}{a+b+c}$$

Dissimilarity between Binary Variables

- Example

Name	Gender	Fever	Cough	Test-1	Test-2	Test-3	Test-4
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	P	N	N	N	N

- gender is a symmetric attribute
- the remaining attributes are asymmetric binary
- let the values Y and P be set to 1, and the value N be set to 0

$$d(jack, mary) = \frac{0+1}{2+0+1} = 0.33$$

$$d(jack, jim) = \frac{1+1}{1+1+1} = 0.67$$

$$d(jim, mary) = \frac{1+2}{1+1+2} = 0.75$$

Nominal Variables

- A generalization of the binary variable in that it can take more than 2 states, e.g., red, yellow, blue, green
- Method 1: Simple matching
  - m: # of matches, p: total # of variables

$$d(i,j) = \frac{p - m}{p}$$

- Method 2: use a large number of binary variables
  - creating a new binary variable for each of the M nominal states

Ordinal Variables

- An ordinal variable can be discrete or continuous
- order is important, e.g., rank
- Can be treated like interval-scaled
  - replacing x_{if} by their rank $r_{if} \in \{1, \dots, M_f\}$
  - map the range of each variable onto [0, 1] by replacing *i*-th object in the *f*-th variable by

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

  - compute the dissimilarity using methods for interval-scaled variables

Variables of Mixed Types

- A database may contain some types of variables
  - symmetric binary, asymmetric binary, nominal, ordinal, interval and ratio.
- One may use a weighted formula to combine their effects.

$$d(i,j) = \frac{\sum_{f=1}^{p} w_f d_f(i,j)}{\sum_{f=1}^{p} w_f}$$

  - f is binary or nominal:

    $d_{ij}^{(f)} = 0$ if $x_{if} = x_{jf}$, or $d_{ij}^{(f)} = 1$
  - f is interval-based: use the normalized distance
  - f is ordinal or ratio-scaled
    - compute ranks r_{if} and $z_{if} = \frac{r_{if} - 1}{M_f - 1}$
    - and treat z_{if} as interval-scaled

Exercises

Variables of Mixed Types

- Given the following table:

Family	Time	Mode	Activity	Satisfaction	With children
A	20	Car	Football, skiing, swimming	2	Y
B	25	Moto	volleyball, football, skiing	1	N
C	30	Bus	Skiing, swimming, volleyball	-2	N
D	45	Train	football, volleyball, swimming	-1	Y

- Calculate the dissimilarity between family

K-means method

- Use the k-means algorithm and Manhattan distance to cluster the following data points into 2 clusters:

Point	1	2	3	4	5	6	7	8
x	1	2	2	1	7	10	8	9
y	10	10	8	9	10	10	7	9

- Suppose that the initial seeds (centers of each cluster) are P5, P7. Run the k-means algorithm for 2 epochs only.

Hierarchical clustering

- Use single link agglomerative clustering to group the data described by the following distance matrix. Show the dendrograms

	A	B	C	D
A	0	1	4	5
B		0	2	6
C			0	3
D				0